BMCS2063 Data Structures and Algorithms

ASSIGNMENT 202505

Declaration

- I confirm that I have read and complied with all the terms and conditions of Tunku Abdul Rahman University of Management and Technology's plagiarism policy.
- I declare that this assignment is free from all forms of plagiarism and for all intents and purposes is my own properly derived work.

Student Name	Student ID	Programme & Tut.Group	Signature
Low Zi Qing	2400272	RDS2S2G1	LOW
Gan Jun Wei	2400372	RDS2S2G1	GJW
Sebastian Tan Huang Zhi	2400311	RDS2S2G1	THZ
Soo Zu Zheng	2400773	RDS2S2G1	SZZ
Teo Geok Woon	2400044	RDS2S2G1	TGW

Note: The submission date and time will be according to the timestamp recorded in Google Classroom for Assignment Submission.

Low Zi Qing

Gan Jun Wei

Sebastian Tan Huang Zhi

Soo Zu Zheng

Teo Geok Woon

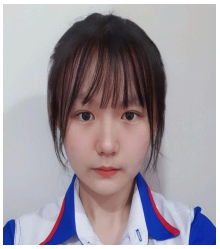


Table of Contents

Part A: TEAM REPORT **3**
1. Abstract Data Type (ADT) 3
1.1 ADT Specification 3
1.2 ADT Implementation 4
a. Interface class (SortedListInterface.java) 4
b. Implementation class (SortedSkipList.java) 4
Part B. INDIVIDUAL REPORT **13**
2. Use of ADTs 13
Teo Geok Woon 13
1. Source Code (Control Class only) 13
2. Screenshots 13
Sebastian Tan Huang Zhi 18
1. Source codes (Control classes) 18
2. Screenshots 30
Low Zi Qing 35
1. Source codes (Control classes) 35
2. Screenshots 35
Gan Jun Wei 35
1. Source codes (Control classes) 35
2. Screenshots 35
Soo Zu Zheng 35
1. Source codes (Control classes) 35
2. Screenshots 35

Part A: TEAM REPORT

1. Abstract Data Type (ADT)

1.1 ADT Specification

SortedList

add (T newEntry)

Description : Adds **newEntry** to the list in sorted order.
Postcondition : **newEntry** has been added to the list in sorted order.

T removeAt (Integer index)

Description : Removes the entry at position **index** within the list.
Precondition : **index** must be between 1 to total entries.
Postcondition : The entry at position **index** has been removed from the list.
Returns : The entry that was removed from the list.

T remove (T anEntry)

Description : Removes the matching entry from the list.
Precondition : **anEntry** is comparable
Postcondition : The entry that is equals to **anEntry** has been removed from the list.
Returns : The entry that was removed from the list.

T get (int index)

Description : Retrieves the entry at position **index** in the list.
Precondition : **index** must be between 1 to total entries.
Postcondition : The sorted list remains the same.
Returns : The entry at the position **index**.

int size ()

Description : Gets the number of entries currently in the list.
Postcondition : The list remains unchanged.
Returns : The number of entries currently in the list.

int getlength()

Description : Gets the number of entries currently in the list.
Postcondition : The list remains unchanged.
Returns : The number of entries currently in the list.

clear()

Description : Removes all entries from the list.
Postcondition : The list is now empty.

boolean isEmpty()

Description : Determines whether the list is empty.

Postcondition : The list remains unchanged.
Returns : **true** if the list is empty, or **false** if not.

boolean contains (T anEntry)

Description : Determines whether the list contains **anEntry**.
Postcondition : The list remains unchanged.
Returns : **true** if **anEntry** is in the list, or **false** if not.

Object[] toArray ()

Description : Creates an array with the contents in the list
Postcondition : The list remains unchanged.
Returns : An array containing the entries in the list.

1.2 ADT Implementation

a. Interface class (SortedListInterface.java)

```
package adt;

/**
 *
 * @author Sebastian Tan & SooZZ
 * @param <T>
 */
public interface SortedListInterface<T> {

    public boolean add(T newEntry);
    public T removeAt(int index);
    public T remove(T anEntry);
    public T get(int index);
    public int size();
    public int getlength();
    public void clear();
    public boolean isEmpty();
    public boolean contains(T anEntry);
    Object[] toArray();
}
```

b. Implementation class (SortedSkipList.java)

```
package adt;

import java.io.Serializable;
import java.util.Iterator;
import java.util.NoSuchElementException;
import java.util.Random;
import java.util.Spliterator;
import java.util.function.Consumer;

/**
 *
```

```
 * @author Sebastian Tan
 * @param <T>
 */
public class SortedSkipList<T extends Comparable<? super T>> implements SortedListInterface<T>, Iterable<T>, Serializable {
    private Node head;
    private Node tail;
    private T negInfinity;  // Sentinel for negative infinity
    private T posInfinity;  // Sentinel for positive infinity
    private int heightOfSkipList = 0;
    private int numberOfEntries = 0;
    public Random random = new Random();

    public SortedSkipList(T negInfinity, T posInfinity) {
        this.negInfinity = negInfinity;
        this.posInfinity = posInfinity;
        head = new Node(negInfinity);
        tail = new Node(posInfinity);
        head.next = tail;
        tail.prev = head;
    }

    @SuppressWarnings("unchecked")
    public SortedSkipList() {
        this.negInfinity = (T) getMinValue();
        this.posInfinity = (T) getMaxValue();
        head = new Node(negInfinity);
        tail = new Node(posInfinity);
        head.next = tail;
        tail.prev = head;
    }

    public boolean add(T newEntry) {
        Node position = search(newEntry);
        Node q;

        int level = -1;
        int numberOfHeads = -1;

        if (position.key.compareTo(newEntry) == 0) {
            return false;//Entry already exists
        }

        do {
            numberOfHeads++;
            level++;
            canIncreaseLevel(level);
            q = position;
            while (position.above == null) {
                position = position.prev;
            }
```

```
            position = position.above;
            q = insertAfterAbove(position, q, newEntry);
        } while (random.nextBoolean() == true);
        numberOfEntries++;
        return true;
    }

    public T remove(T anEntry) {
        Node nodeToBeRemoved = search(anEntry);

        if (nodeToBeRemoved.key.compareTo(anEntry) != 0) {
            return null; //Entry not found
        }
        removeReferencesToNode(nodeToBeRemoved);

        while(nodeToBeRemoved != null) {
            removeReferencesToNode(nodeToBeRemoved);

            if (nodeToBeRemoved.above != null) {
                nodeToBeRemoved = nodeToBeRemoved.above;
            } else {
                break;
            }
        }

        numberOfEntries--;
        return nodeToBeRemoved.key;
    }

    public void clear() {
        heightOfSkipList = 0;
        numberOfEntries = 0;
        head = new Node(negInfinity);
        tail = new Node(posInfinity);
        head.next = tail;
        tail.prev = head;
    }

    public boolean contains(T anEntry) {
        Node position = search(anEntry);
        return position.key.compareTo(anEntry) == 0;
    }

    public int getLength() {
        return numberOfEntries;
    }

    public boolean isEmpty() {
        return numberOfEntries == 0;
    }

```

```
    public void printList() {
        StringBuilder sb = new StringBuilder();
        sb.append("\nSkipList starting with top-left most node.\n");

        Node starting = head;

        Node highestLevel = starting;
        int level = heightOfSkipList;

        while (highestLevel != null) {
            sb.append("\nLevel: " + level + "\n");

            while (starting != null) {
                sb.append(starting.key);

                if (starting.next != null) {
                    sb.append(" : ");
                }

                starting = starting.next;
            }
            highestLevel = highestLevel.below;
            starting = highestLevel;
            level--;
        }
        System.out.println(sb.toString());
    }

    public void printContents() {
        StringBuilder sb = new StringBuilder();

        Node current = getFirstDataNode();

        while (current != null && current != tail && current.key != posInfinity) {
            sb.append(current.key);

            if (current.next != null && current.next != tail && current.next.key !=
posInfinity) {
                sb.append("\n");
            }

            current = current.next;
        }

        System.out.println(sb.toString());
    }

    // Helper methods to get min/max values for common types
    private Comparable<?> getMinValue() {
        return Integer.MIN_VALUE;
    }
```

```java

    private Comparable<?> getMaxValue() {
        return Integer.MAX_VALUE;
    }

    public Node search(T anEntry) {
        Node n = head;

        while (n.below != null) {
            n =  n.below;
            while (anEntry.compareTo(n.next.key)>=0) {
                n = n.next;
            }
        }
        return n;
    }

    public T get(T anEntry) {
        Node n = head;

        while (n.below != null) {
            n =  n.below;
            while (anEntry.compareTo(n.next.key)>=0) {
                n = n.next;
            }
        }
        return n.key;
    }

    private void removeReferencesToNode(Node nodeToBeRemoved) {
        Node afterNodeToBeRemoved = nodeToBeRemoved.next;
        Node beforeNodeToBeRemoved = nodeToBeRemoved.prev;

        beforeNodeToBeRemoved.next = afterNodeToBeRemoved;
        afterNodeToBeRemoved.prev = beforeNodeToBeRemoved;
    }

    private void canIncreaseLevel(int level) {
        if (level >= heightOfSkipList) {
            heightOfSkipList++;
            addEmptyLevel();
        }
    }

    private void addEmptyLevel() {
        Node newHeadNode = new Node(negInfinity);
        Node newTailNode = new Node(posInfinity);

        newHeadNode.next = newTailNode;
        newHeadNode.below = head;
        newTailNode.prev = newHeadNode;
```

```
        newTailNode.below = tail;

        head.above = newHeadNode;
        tail.above = newTailNode;

        head = newHeadNode;
        tail = newTailNode;
    }

    private Node insertAfterAbove(Node position, Node q, T key) {
        Node newNode = new Node(key);
        Node nodeBeforeNewNode = position.below.below;

        setBeforeAfterReferences(q, newNode);
        setAboveAndBelowReferences(position, key, newNode, nodeBeforeNewNode);

        return newNode;
    }

    private void setBeforeAfterReferences(Node q, Node newNode) {
        newNode.next = q.next;
        newNode.prev = q;
        q.next.prev = newNode;
        q.next = newNode;
    }

    private void setAboveAndBelowReferences(Node position, T key, Node newNode, Node nodeBeforeNewNode) {
        if (nodeBeforeNewNode != null) {
            while (true) {
                if (nodeBeforeNewNode.next.key.compareTo(key) != 0) {
                    nodeBeforeNewNode = nodeBeforeNewNode.next;
                } else {
                    break;
                }
            }

            newNode.below = nodeBeforeNewNode.next;
            nodeBeforeNewNode.next.above = newNode;
        }

        if (position != null) {
            if (position.next.key.compareTo(key) == 0) {
                newNode.above = position.next;
            }
        }
    }


    private Node getFirstDataNode() {
```

```
    Node current = head;
    // Go to bottom level
    while (current.below != null) {
        current = current.below;
    }
    // Skip negative infinity sentinel
    return current.next;
}

@Override
public int getlength() {
    return numberOfEntries;
}

@Override
public Iterator<T> iterator() {
    return new SkipListIterator();
}

@Override
public void forEach(Consumer<? super T> action) {
    Iterable.super.forEach(action);
}

@Override
public Spliterator<T> spliterator() {
    return Iterable.super.spliterator();
}

private class SkipListIterator implements Iterator<T> {
    private Node current;

    public SkipListIterator() {
        current = getFirstDataNode();
    }

    @Override
    public boolean hasNext() {
        return current != null &&
               current != tail &&
               !current.key.equals(posInfinity);
    }

    @Override
    public T next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        T data = current.key;
        current = current.next;
        return data;
```

```
    }

    @Override
    public void remove() {
        throw new UnsupportedOperationException("Remove not supported");
    }
}

private class Node implements Serializable{
    private Node above;
    private Node below;
    private Node next;
    private Node prev;
    private T key;

    public Node(T key) {
        this.key = key;
        this.above = null;
        this.below = null;
        this.next = null;
        this.prev = null;
    }
}

public T removeAt(int index) {
    if (index < 0 || index >= numberOfEntries) {
        return null;
    }

    Node current = getFirstDataNode();
    for (int i = 0; i < index; i++) {
        current = current.next;
    }

    T removedData = current.key;
    remove(removedData);
    return removedData;
}

public T get(int index) {
    if (index < 0 || index >= numberOfEntries) {
        return null;
    }

    Node current = getFirstDataNode();
    for (int i = 0; i < index; i++) {
        current = current.next;
    }

    return current.key;
}
```

```

    public int size() {
        return numberOfEntries;
    }

    @Override
    public Object[] toArray() {
        Object[] array = new Object[numberOfEntries];
        Node current = getFirstDataNode();
        int index = 0;

        while (current != null && current != tail &&
!current.key.equals(posInfinity)) {
            array[index++] = current.key;
            current = current.next;
        }

        return array;
    }

}
```

Part B. INDIVIDUAL REPORT

2. Use of ADTs

Name	Student ID	Prog / Tut.Grp	Signature
Teo Geok Woon	2400044	RDS2S2G1	TGW

user interface input and outputs for core functions

Module name: Patient Management Module

1. Source Code (Control Class only)

<Print html from Netbean and copy, paste the code>

a. Ensure that the author of each class is stated in the comment section at the beginning of each class,

b. Highlight Font as Yellow for the lines of code that involve either the declaration / creation of collection ADT objects, and the invocation of collection ADT methods(called).

```
package control;

/** Author: Teo Geok Woon */

import adt.ArrayQueue;
import adt.LinkedList;
import adt.ListInterface;
import entity.Patient;
import dao.PatientDAO;
import dao.ActivityDAO;

import java.io.Serializable;
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalDateTime;

public class MaintainPatient {
    public static class Pair<K,V> implements Serializable {
        private static final long serialVersionUID = 1L;
        public final K key; public V value;
```

```
        public Pair(K k, V v) { key = k; value = v; }
    }
    public static class DateWaits implements Serializable {
        private static final long serialVersionUID = 1L;
        public LocalDate date;
        public ListInterface<Long> waits = new LinkedList<>();
        public DateWaits(LocalDate d) { date = d; }
    }

    // ---------- data ----------
    private ListInterface<Patient> records = new LinkedList<>();
    private final ListInterface<Registration> registrations = new LinkedList<>();
    private final ListInterface<DeletionEvent> deletions = new LinkedList<>();
    private final ListInterface<Pair<String, LocalDateTime>> queueEnqueueTimes = new LinkedList<>();
    private final ListInterface<DateWaits> waitHistoryByDate = new LinkedList<>();
    private final ListInterface<QueueEvent> queueEvents = new LinkedList<>();

    private final ArrayQueue<Patient> waitingQueue = new ArrayQueue<>();
    private final ArrayQueue<Patient> emergencyQueue = new ArrayQueue<>();
    private final PatientDAO patientDAO = new dao.PatientDAO();
    private final ActivityDAO activityDAO = new ActivityDAO();
    private int idCounter = 1;

    public ListInterface<Registration> getRegistrations() { return registrations; }
    public ListInterface<QueueEvent>   getQueueEvents()  { return queueEvents; }
    public ListInterface<DeletionEvent> getDeletions()   { return deletions; }
    public ListInterface<Pair<String, LocalDateTime>> getQueueEnqueueTimesList() { return queueEnqueueTimes; }
    public ListInterface<DateWaits> getWaitHistoryByDateList() { return waitHistoryByDate; }

    public MaintainPatient() {
        records = patientDAO.retrieveFromFile();
        reseedIdCounterFromRecords();
        loadActivityFromDisk();
```

```
}

public static <T> int sizeOf(ArrayQueue<T> q) {
    int n = 0;
    java.util.Iterator<T> it = q.getIterator();
    while (it.hasNext()) { it.next(); n++; }
    return n;
}

private static <T> boolean queueContains(ArrayQueue<T> q, T target) {
    java.util.Iterator<T> it = q.getIterator();
    while (it.hasNext()) if (java.util.Objects.equals(it.next(), target)) return true;
    return false;
}

private static <T> boolean removeFromQueue(ArrayQueue<T> q, T target) {
    int n = sizeOf(q);
    if (n == 0) return false;
    ArrayQueue<T> tmp = new ArrayQueue<>(n);
    boolean removed = false;
    while (!q.isEmpty()) {
        T x = q.dequeue();
        if (!removed && java.util.Objects.equals(x, target)) removed = true;
        else tmp.enqueue(x);
    }
    while (!tmp.isEmpty()) q.enqueue(tmp.dequeue());
    return removed;
}

public static class Registration implements Serializable {
    private static final long serialVersionUID = 1L;
    public final String patientId, name, ic, phone, address, notes, gender, department,
appointmentType;
    public final int age;
    public final boolean emergency;
    public final LocalDateTime time;
```

```
    Registration(Patient p) {
        this.patientId = p.getPatientId();
        this.name = p.getName();
        this.ic = p.getIc();
        this.age = p.getAge();
        this.phone = p.getPhone();
        this.address = p.getAddress();
        this.notes = p.getNotes();
        this.gender = p.getGender();
        this.department = p.getDepartment();
        this.appointmentType = p.getAppointmentType();
        this.emergency = p.getEmergency();
        this.time = LocalDateTime.now();
    }
}

public boolean addPatient(Patient p) {
    if (p == null) return false;
    if (icExists(p.getIc())) return false;
    if (records.contains(p)) return false;
    boolean ok = records.add(p);
    if (ok) {
        registrations.add(new Registration(p));
        saveNow();
        saveActivity();
    }
    return ok;
}

public enum UpdateField { NAME, IC, AGE, PHONE, ADDRESS, NOTES, GENDER, DEPARTMENT, APPOINTMENT_TYPE, EMERGENCY }

public boolean updateField(String patientId, UpdateField field, Object value) {
    Patient p = findPatientById(patientId);
    if (p == null || field == null) return false;
    switch (field) {
```

```
            case NAME:        if (value instanceof String) p.setName((String) value); else return false; break;
            case IC:          if (value instanceof String) { String newIc = (String) value;
                              if (icExistsForOther(newIc, patientId)) return false; p.setIc(newIc); } else return false; break;
            case AGE:         if (value instanceof Integer) p.setAge((Integer) value); else return false; break;
            case PHONE:       if (value instanceof String) p.setPhone((String) value); else return false; break;
            case ADDRESS:     if (value instanceof String) p.setAddress((String) value); else return false; break;
            case NOTES:       if (value instanceof String) p.setNotes((String) value); else return false; break;
            case GENDER:      if (value instanceof String) p.setGender((String) value); else return false; break;
            case DEPARTMENT:  if (value instanceof String) p.setDepartment((String) value); else return false; break;
            case APPOINTMENT_TYPE:
                              if (value instanceof String) p.setAppointmentType((String) value); else return false; break;
            case EMERGENCY:   if (value instanceof Boolean) p.setEmergency((Boolean) value); else return false; break;
            default: return false;
        }
        saveNow();
        return true;
    }

    public boolean updatePatient(String patientId, String name, String ic, Integer age, String phone, String address,
                                 String notes, String gender, String department, String appointmentType, Boolean emergency) {
        boolean changed = false;
        if (name != null)       changed |= updateField(patientId, UpdateField.NAME, name);
        if (ic != null)         changed |= updateField(patientId, UpdateField.IC, ic);
        if (age != null)        changed |= updateField(patientId, UpdateField.AGE, age);
        if (phone != null)      changed |= updateField(patientId, UpdateField.PHONE, phone);
        if (address != null)    changed |= updateField(patientId, UpdateField.ADDRESS, address);
```

```
150         if (notes != null)       changed |= updateField(patientId, UpdateField.NOTES, notes);
151         if (gender != null)      changed |= updateField(patientId, UpdateField.GENDER,
gender);
152         if (department != null)  changed |= updateField(patientId,
UpdateField.DEPARTMENT, department);
153         if (appointmentType != null) changed |= updateField(patientId,
UpdateField.APPOINTMENT_TYPE, appointmentType);
154         if (emergency != null)   changed |= updateField(patientId,
UpdateField.EMERGENCY, emergency);
155         return changed;
156     }
157
158     public boolean deletePatient(String patientId) {
159         Patient p = findPatientById(patientId);
160         if (p == null) return false;
161         removeFromQueue(emergencyQueue, p);
162         removeFromQueue(waitingQueue, p);
163         timeRemove(p.getPatientId());
164         boolean removed = records.remove(p);
165         if (removed) {
166             deletions.add(new DeletionEvent(patientId, LocalDateTime.now()));
167             saveNow();
168             saveActivity();
169         }
170         return removed;
171     }
172
173     public static class QueueEvent implements Serializable {
174         private static final long serialVersionUID = 1L;
175         public final String patientId;
176         public final LocalDateTime time;
177         QueueEvent(String patientId, LocalDateTime time) { this.patientId = patientId;
this.time = time; }
178     }
179
180     public static class DeletionEvent implements Serializable {
181         private static final long serialVersionUID = 1L;
```

```
182         public final String patientId;
183         public final LocalDateTime time;
184         DeletionEvent(String patientId, LocalDateTime time) { this.patientId = patientId;
this.time = time; }
185     }
186
187     private void reseedIdCounterFromRecords() {
188         int max = 0;
189         for (int i = 0; i < records.size(); i++) {
190             Patient p = records.get(i);
191             if (p == null) continue;
192             String id = p.getPatientId();
193             if (id != null && id.matches("^P\\d{4}$")) {
194                 int n = Integer.parseInt(id.substring(1));
195                 if (n > max) max = n;
196             }
197         }
198         idCounter = max + 1;
199     }
200
201     public String generatePatientId() {
202         String id;
203         do { id = String.format("P%04d", idCounter++); } while (findPatientById(id) !=
null);
204         return id;
205     }
206
207     private void logRegistration(Patient p) { registrations.add(new Registration(p));
saveActivity(); }
208
209     public Patient findPatientById(String patientId) {
210         for (int i = 0; i < records.size(); i++) {
211             Patient p = records.get(i);
212             if (p != null && p.getPatientId().equals(patientId)) return p;
213         }
214         return null;
```

```
}

private boolean icExistsForOther(String ic, String selfId) {
    if (ic == null) return false;
    String key = ic.replaceAll("[^0-9]", "");
    for (int i = 0; i < records.size(); i++) {
        Patient p = records.get(i);
        if (p == null || p.getIc() == null) continue;
        String id = p.getPatientId();
        if (selfId != null && selfId.equals(id)) continue;
        String icDigits = p.getIc().replaceAll("[^0-9]", "");
        if (icDigits.equals(key)) return true;
    }
    return false;
}

public ListInterface<Patient> listAllPatients() {
    records = patientDAO.retrieveFromFile();
    return records;
}

public boolean enqueuePatient(String patientId) {
    Patient p = findPatientById(patientId);
    if (p == null) return false;
    if (queueContains(waitingQueue, p) || queueContains(emergencyQueue, p)) return false;

    if (p.getEmergency()) emergencyQueue.enqueue(p);
    else waitingQueue.enqueue(p);

    LocalDateTime now = LocalDateTime.now();
    timePut(p.getPatientId(), now);
    queueEvents.add(new QueueEvent(p.getPatientId(), now));
    saveActivity();
    return true;
}
```

```java

    public Patient dequeuePatient() {
        Patient served = null;
        if (!emergencyQueue.isEmpty()) served = emergencyQueue.dequeue();
        else if (!waitingQueue.isEmpty()) served = waitingQueue.dequeue();
        else return null;

        LocalDateTime enq = timeGet(served.getPatientId());
        timeRemove(served.getPatientId());
        if (enq != null) {
            long ms = Duration.between(enq, LocalDateTime.now()).toMillis();
            LocalDate day = LocalDate.now();
            waitsFor(day).add(ms);
            saveActivity();
        }
        return served;
    }

    public Patient peekQueue() {
        if (!emergencyQueue.isEmpty()) return emergencyQueue.getFront();
        if (!waitingQueue.isEmpty()) return waitingQueue.getFront();
        return null;
    }

    public ArrayQueue<Patient> getEmergencyQueueSnapshot() {
        ArrayQueue<Patient> snap = new ArrayQueue<>();
        java.util.Iterator<Patient> it = emergencyQueue.getIterator();
        while (it.hasNext()) snap.enqueue(it.next());
        return snap;
    }

    public ArrayQueue<Patient> getNormalQueueSnapshot() {
        ArrayQueue<Patient> snap = new ArrayQueue<>();
        java.util.Iterator<Patient> it = waitingQueue.getIterator();
        while (it.hasNext()) snap.enqueue(it.next());
```

```
285         return snap;
286     }
287
288     public ArrayQueue<Patient> searchByIc(String query) {
289         ArrayQueue<Patient> result = new ArrayQueue<>();
290         if (query == null) return result;
291         String key = query.trim().replaceAll("[^0-9]", "");
292         if (key.isEmpty()) return result;
293         for (int i = 0; i < records.size(); i++) {
294             Patient p = records.get(i);
295             if (p == null) continue;
296             String ic = p.getIc();
297             if (ic == null) continue;
298             String icDigits = ic.replaceAll("[^0-9]", "");
299             if (icDigits.contains(key)) result.enqueue(p);
300         }
301         return result;
302     }
303
304     public Patient findPatientByIcExact(String ic) {
305         if (ic == null) return null;
306         String key = ic.replaceAll("[^0-9]", "");
307         for (int i = 0; i < records.size(); i++) {
308             Patient p = records.get(i);
309             if (p == null || p.getIc() == null) continue;
310             String icDigits = p.getIc().replaceAll("[^0-9]", "");
311             if (icDigits.equals(key)) return p;
312         }
313         return null;
314     }
315
316     public void logReturningRegistration(String patientId) {
317         Patient p = findPatientById(patientId);
318         if (p != null) registrations.add(new Registration(p));
319     }
```

```
    public boolean icExists(String ic) { return findPatientByIcExact(ic) != null; }

    private void loadActivityFromDisk() {
        ActivityDAO.ActivitySnapshot snap = activityDAO.retrieveFromFile();

        this.registrations.clear();
        for (int i = 0; i < snap.registrations.size(); i++) this.registrations.add(snap.registrations.get(i));

        this.deletions.clear();
        for (int i = 0; i < snap.deletions.size(); i++) this.deletions.add(snap.deletions.get(i));

        this.queueEvents.clear();
        for (int i = 0; i < snap.queueEvents.size(); i++) this.queueEvents.add(snap.queueEvents.get(i));

        this.waitHistoryByDate.clear();
        for (int i = 0; i < snap.waitHistoryByDate.size(); i++) this.waitHistoryByDate.add(snap.waitHistoryByDate.get(i));
    }

    private void saveNow() { patientDAO.saveToFile(records); }

    private void saveActivity() {
        ActivityDAO.ActivitySnapshot out = new ActivityDAO.ActivitySnapshot();

        for (int i = 0; i < this.registrations.size(); i++) out.registrations.add(this.registrations.get(i));
        for (int i = 0; i < this.deletions.size(); i++)     out.deletions.add(this.deletions.get(i));
        for (int i = 0; i < this.queueEvents.size(); i++) out.queueEvents.add(this.queueEvents.get(i));

        // deep copy waits
        for (int i = 0; i < this.waitHistoryByDate.size(); i++) {
            DateWaits src = this.waitHistoryByDate.get(i);
            DateWaits dst = new DateWaits(src.date);
```

```
352            for (int j = 0; j < src.waits.size(); j++) dst.waits.add(src.waits.get(j));
353            out.waitHistoryByDate.add(dst);
354        }
355        activityDAO.saveToFile(out);
356    }
357
358    // ---------- "map-like" helpers ----------
359    private LocalDateTime timeGet(String pid) {
360        for (int i = 0; i < queueEnqueueTimes.size(); i++) {
361            Pair<String, LocalDateTime> p = queueEnqueueTimes.get(i);
362            if (p.key.equals(pid)) return p.value;
363        }
364        return null;
365    }
366    private void timePut(String pid, LocalDateTime t) {
367        for (int i = 0; i < queueEnqueueTimes.size(); i++) {
368            if (queueEnqueueTimes.get(i).key.equals(pid)) { queueEnqueueTimes.remove(i);
break; }
369        }
370        queueEnqueueTimes.add(new Pair<>(pid, t));
371    }
372    private void timeRemove(String pid) {
373        for (int i = 0; i < queueEnqueueTimes.size(); i++) {
374            if (queueEnqueueTimes.get(i).key.equals(pid)) { queueEnqueueTimes.remove(i);
return; }
375        }
376    }
377    private ListInterface<Long> waitsFor(LocalDate d) {
378        for (int i = 0; i < waitHistoryByDate.size(); i++) {
379            DateWaits dw = waitHistoryByDate.get(i);
380            if (dw.date.equals(d)) return dw.waits;
381        }
382        DateWaits nw = new DateWaits(d);
383        waitHistoryByDate.add(nw);
384        return nw.waits;
385    }
```

```
// ==================== SUMMARY REPORT STRUCTS/LOGIC ====================
public static class DailyRegistrationStats {
    public int totalRegistrationEvents;
    public int distinctRegistered;
    public int newPatients;
    public int returningPatients;

    public int male, female, genderUnknown;
    public int children, adults, seniors;

    // Department counts without JCF:
    public ListInterface<Pair<String,Integer>> deptCounts = new LinkedList<>();

    public int regWalkIn, regAppointment, regUnknown;

    public int totalEnqueueEvents;
    public int distinctEnqueuedPatients;
    public int enqWalkIn, enqAppointment;

    public int[] hourlyEnqueues = new int[24];
    public int busiestHour = -1, busiestHourCount = 0;
}

public static class QueueStatusSnapshot {
    public int queueLength;
    public int emergencyCount;
    public long avgWaitMinutesCurrent;

    public static class DayWait {
        public LocalDate date; public int served; public long avgWaitMinutes;
    }
    public ListInterface<DayWait> last7Days = new LinkedList<>();

    public static class Row {
```

```
        public int position; public String name; public int age; public String department;
        public boolean emergency; public long minutesWait;
    }
    public ListInterface<Row> rows = new LinkedList<>();
}

private static String digits(String s){ return (s==null) ? "" : s.replaceAll("[^0-9]",""); }

public DailyRegistrationStats generateDailyRegistrationStats(LocalDate day) {
    DailyRegistrationStats s = new DailyRegistrationStats();

    // : earliest registration date PER IC across all time
    ListInterface<Pair<String, LocalDate>> earliestByIc = new LinkedList<>();
    for (int i = 0; i < registrations.size(); i++) {
        Registration r = registrations.get(i);
        String icKey = digits(r.ic);
        int idx = indexOfKey(earliestByIc, icKey);
        if (idx < 0) {
            earliestByIc.add(new Pair<>(icKey, r.time.toLocalDate()));
        } else {
            LocalDate cur = earliestByIc.get(idx).value;
            LocalDate cand = r.time.toLocalDate();
            if (cand.isBefore(cur)) earliestByIc.get(idx).value = cand;
        }
    }

    // count today's events (by IC)
    ListInterface<String> distinctIcToday = new LinkedList<>();
    ListInterface<Pair<String,Integer>> countByIcToday = new LinkedList<>();

    for (int i = 0; i < registrations.size(); i++) {
        Registration r = registrations.get(i);
        if (!r.time.toLocalDate().equals(day)) continue;

        s.totalRegistrationEvents++;
```

```
456
457         // distinct ICs today
458         String icKey = digits(r.ic);
459         if (!distinctIcToday.contains(icKey)) distinctIcToday.add(icKey);
460
461         // per-IC frequency today
462         int cIdx = indexOfKey(countByIcToday, icKey);
463         if (cIdx < 0) countByIcToday.add(new Pair<>(icKey, 1));
464         else countByIcToday.get(cIdx).value = countByIcToday.get(cIdx).value + 1;
465
466         // per-event breakdowns (gender/age/dept/type)
467         String g = r.gender == null ? "" : r.gender.trim();
468         if ("male".equalsIgnoreCase(g)) s.male++;
469         else if ("female".equalsIgnoreCase(g)) s.female++;
470         else s.genderUnknown++;
471
472         int age = r.age;
473         if (age < 18) s.children++;
474         else if (age >= 65) s.seniors++;
475         else s.adults++;
476
477         String dept = (r.department == null || r.department.isBlank()) ? "(Unknown)" :
r.department;
478         int dIdx = indexOfKey(s.deptCounts, dept);
479         if (dIdx < 0) s.deptCounts.add(new Pair<>(dept, 1));
480         else s.deptCounts.get(dIdx).value = s.deptCounts.get(dIdx).value + 1;
481
482         String at = r.appointmentType == null ? "" : r.appointmentType.trim();
483         if ("walk-in".equalsIgnoreCase(at)) s.regWalkIn++;
484         else if ("appointment".equalsIgnoreCase(at)) s.regAppointment++;
485         else s.regUnknown++;
486     }
487
488     // distinct registered today = distinct ICs that appeared today
489     s.distinctRegistered = distinctIcToday.size();
490
```

```
491     // new patients today = ICs whose earliest-ever registration date == today
492     int newCount = 0;
493     for (int i = 0; i < distinctIcToday.size(); i++) {
494         String icKey = distinctIcToday.get(i);
495         int eIdx = indexOfKey(earliestByIc, icKey);
496         if (eIdx >= 0 && day.equals(earliestByIc.get(eIdx).value)) newCount++;
497     }
498     s.newPatients = newCount;
499
500     // returning patients today = ICs with >1 registrations today (your requirement)
501     int returningDistinct = 0;
502     for (int i = 0; i < countByIcToday.size(); i++) {
503         if (countByIcToday.get(i).value > 1) returningDistinct++;
504     }
505     s.returningPatients = returningDistinct;
506
507     ListInterface<String> enqDistinct = new LinkedList<>();
508     for (int i = 0; i < queueEvents.size(); i++) {
509         QueueEvent e = queueEvents.get(i);
510         if (!e.time.toLocalDate().equals(day)) continue;
511         s.totalEnqueueEvents++;
512         s.hourlyEnqueues[e.time.getHour()]++;
513         if (!enqDistinct.contains(e.patientId)) enqDistinct.add(e.patientId);
514
515         Patient p = findPatientById(e.patientId);
516         if (p != null) {
517             String t = p.getAppointmentType();
518             if ("walk-in".equalsIgnoreCase(t)) s.enqWalkIn++;
519             else if ("appointment".equalsIgnoreCase(t)) s.enqAppointment++;
520         }
521     }
522     s.distinctEnqueuedPatients = enqDistinct.size();
523
524     for (int h = 0; h < 24; h++) {
525         if (s.hourlyEnqueues[h] > s.busiestHourCount) { s.busiestHourCount =
s.hourlyEnqueues[h]; s.busiestHour = h; }
```

```
        }
        return s;
    }

    private static <V> int indexOfKey(ListInterface<Pair<String, V>> list, String key) {
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i).key.equals(key)) return i;
        }
        return -1;
    }

    public QueueStatusSnapshot generateQueueStatusSnapshot() {
        QueueStatusSnapshot q = new QueueStatusSnapshot();

        ArrayQueue<Patient> em = getEmergencyQueueSnapshot();
        ArrayQueue<Patient> nm = getNormalQueueSnapshot();
        q.emergencyCount = sizeOf(em);
        q.queueLength = q.emergencyCount + sizeOf(nm);

        long totalMs = 0L; int count = 0;
        java.util.Iterator<Patient> it1 = em.getIterator();
        while (it1.hasNext()) {
            Patient p = it1.next();
            LocalDateTime t = timeGet(p.getPatientId());
            if (t != null) { totalMs += Duration.between(t, LocalDateTime.now()).toMillis(); count++; }
        }
        java.util.Iterator<Patient> it2 = nm.getIterator();
        while (it2.hasNext()) {
            Patient p = it2.next();
            LocalDateTime t = timeGet(p.getPatientId());
            if (t != null) { totalMs += Duration.between(t, LocalDateTime.now()).toMillis(); count++; }
        }
        q.avgWaitMinutesCurrent = (count == 0) ? 0 : (totalMs / count) / 1000 / 60;

```

```
        LocalDate today = LocalDate.now();
        for (int back = 6; back >= 0; back--) {
            LocalDate d = today.minusDays(back);
            ListInterface<Long> waits = waitsFor(d); // returns existing or creates empty
            long avgMs = 0; int served = waits.size();
            if (served > 0) {
                long sum = 0; for (int i = 0; i < waits.size(); i++) sum += waits.get(i);
                avgMs = sum / served;
            }
            QueueStatusSnapshot.DayWait row = new QueueStatusSnapshot.DayWait();
            row.date = d; row.served = served; row.avgWaitMinutes = (avgMs / 1000) / 60;
            q.last7Days.add(row);
        }

        int pos = 1;
        it1 = em.getIterator();
        while (it1.hasNext()) {
            Patient p = it1.next();
            QueueStatusSnapshot.Row r = new QueueStatusSnapshot.Row();
            r.position = pos++; r.name = p.getName(); r.age = p.getAge();
            r.department = (p.getDepartment()==null || p.getDepartment().isBlank()) ? "-" : p.getDepartment();
            r.emergency = true;
            LocalDateTime t = timeGet(p.getPatientId());
            r.minutesWait = (t == null) ? 0 : Duration.between(t, LocalDateTime.now()).toMinutes();
            q.rows.add(r);
        }
        it2 = nm.getIterator();
        while (it2.hasNext()) {
            Patient p = it2.next();
            QueueStatusSnapshot.Row r = new QueueStatusSnapshot.Row();
            r.position = pos++; r.name = p.getName(); r.age = p.getAge();
            r.department = (p.getDepartment()==null || p.getDepartment().isBlank()) ? "-" : p.getDepartment();
            r.emergency = false;
```

```
            LocalDateTime t = timeGet(p.getPatientId());
            r.minutesWait = (t == null) ? 0 : Duration.between(t, LocalDateTime.now()).toMinutes();
            q.rows.add(r);
        }
        return q;
    }
}
```

2. Screenshots

Includes screenshot of of your modules. Label your screenshots clearly.

```
=== Patient Management Module ===

Menu:
1) Register new patient
2) View all patients
3) Search patient by ID
4) Search patients by IC Number
5) Update patient record
6) Delete patient record
7) Enqueue patient to waiting queue
8) Dequeue next patient
9) View waiting queue
10) Summary Report
0) Exit
Enter choice: 1
```

This is the menu of the Patient Management Module.

```
Enter choice: 1
--- Register Patient ---
Enter IC number (without dash): 010214015836
Enter patient name: Tan Ah San
Enter patient age: 24
Enter gender (Male/Female): Male
Enter department: Physiotherapy
Enter patient phone number (01xxxxxxxx): 0123344556
Enter patient Address: Setapak
Enter notes (optional): Therapies to restore mobility
Appointment type (Walk-in/Appointment): Appointment
Emergency case? (y/n): n
Registered. Patient ID: P0010
```

The function allows users to register a new patient after a key in all the patient information.

```
--- Register Patient ---
Enter IC number (without dash): 010214015836
This IC is already registered as: Tan Ah San (ID P0010)
1) Use existing record and enqueue
2) Update details then enqueue
3) Cancel
Choice:
```

If the new registered patient's identity card number already exists in the system, users can choose to use the existing information of the patient directly or update the patient information.

```
-- ----
Enter choice: 2
--- All Patients ---
ID:P0001 | Name:Ahmad Ali | IC:941105145123 | Age:30 | Gender:Male | Faculty:Cardiology | Phone:0123456789 | Addr:Kuala Lumpur | Type:Walk-in | Emergency:false | Notes:
ID:P0002 | Name:Siti Nur | IC:991014100824 | Age:25 | Gender:Male | Faculty:Orthopedics | Phone:0122345678 | Addr:Petaling Jaya | Type:Appointment | Emergency:true | Notes:
ID:P0003 | Name:Lim Wei | IC:850322077745 | Age:40 | Gender:Male | Faculty:Pediatrics | Phone:0123451234 | Addr:George Town | Type:Walk-in | Emergency:false | Notes:
ID:P0004 | Name:Xiao Meng | IC:010203040506 | Age:19 | Gender:Male | Faculty:Psychiatry | Phone:0123456789 | Addr:Melaka | Type:Walk-in | Emergency:true | Notes:
ID:P0005 | Name:Abu | IC:031223130027 | Age:10 | Gender:Male | Faculty:Urology | Phone:0123123456 | Addr:Setapak | Type:Walk-in | Emergency:true | Notes:
ID:P0006 | Name:Ahmin | IC:020329140029 | Age:24 | Gender:Female | Faculty:General Medical | Phone:0123489563 | Addr:Muar | Type:Walk-in | Emergency:false | Notes:
ID:P0007 | Name:Tan Zhi Hui | IC:051203010039 | Age:20 | Gender:Female | Faculty:Anesthesiology | Phone:0123234456 | Addr:Simpang | Type:Walk-in | Emergency:false | Notes:
ID:P0008 | Name:Tan Mei Mei | IC:091228012345 | Age:34 | Gender:Female | Faculty:Ophthalmology | Phone:0123234345 | Addr:PV15 | Type:Walk-in | Emergency:false | Notes:
ID:P0009 | Name:Bear | IC:021324253647 | Age:3 | Gender:Female | Faculty:vaccination | Phone:0122334567 | Addr:Jalan Pisang | Type:Walk-in | Emergency:false | Notes:
ID:P0010 | Name:Tan Ah San | IC:010214015836 | Age:25 | Gender:Male | Faculty:Physiotherapy | Phone:0123344556 | Addr:Setapak | Type:Appointment | Emergency:false | Notes:Therapies to restore mobility
```

By choosing 2, users are able to view all the registered patients.

```
Enter choice: 3
Enter patient ID: P0001
ID:P0001 | Name:Ahmad Ali | IC:941105145123 | Age:30 | Gender:Male | Faculty:Cardiology | Phone:0123456789 | Addr:Kuala Lumpur | Type:Walk-in | Emergency:false | Notes:
```

Users are allowed to search the patient information they want by entering the patient's ID.

```
Enter choice: 4
Enter IC search term: 010214015836
ID:P0010 | Name:Tan Ah San | IC:010214015836 | Age:25 | Gender:Male | Faculty:Physiotherapy | Phone:0123344556 | Addr:Setapak | Type:Appointment | Emergency:false | Notes:Therapies to restore mobility
```

Users can also search the patient information they want by entering the patient's Identity card number.

```
Enter choice: 5
Enter patient ID to update: P0001

--- Update Patient (P0001 | Ahmad Ali) ---
1) Name
2) IC
3) Age
4) Phone
5) Address
6) Notes
7) Gender
8) Department
9) Appointment Type
10) Emergency
0) Done
Choose field:
```

Users are allowed to manage patient information by updating any field of patient information.

```
Enter choice: 6
Enter patient ID to delete: P0010
Deleted.
```

Users can delete patient information by entering the patient ID.

```
Enter choice: 7
Enter patient ID to enqueue: P0002
Enqueued.
```

Users can enqueue patients for waiting their turn to consultation.

```
Enter choice: 9
--- Queue ---
Emergency Queue
===============
1) Siti Nur

Normal Queue
============
2) Ahmad Ali
```

When a patient is flagged as an emergency, the system routes them to the emergency

queue. This queue is processed ahead of the normal queue, so emergency patients are served first even if others were queued earlier.

```
Enter choice: 8
Dequeued: ID:P0002 | Name:Siti Nur | IC:991014100824 | Age:25 | Gender:Male | Faculty:Orthopedics | Phone:0122345678 | Addr:Petaling Jaya | Type:Appointment | Emergency:true | Notes:
```

The system dequeue patients when it is their turn for consultation.

```
==========================================================================================
                                          REPORTS
==========================================================================================
1) Daily Patient Registration Report
2) Patient Queue Status Report
0) Back to Main Menu
Choose: 1
Use today? (Y/n): n
Enter date (yyyy-MM-dd): 2025-08-30
==========================================================================================
                                PATIENT MANAGEMENT SUBSYSTEM
                             DAILY PATIENT REGISTRATION REPORT
==========================================================================================
Generated at: 2025-08-31 00:04:21
------------------------------------------------------------------------------------------
Date: 2025-08-30
------------------------------------------------------------------------------------------
Total REGISTRATION events today            : 4
Total DISTINCT patients registered today   : 3
New patient registrations today            : 1
Returning registrations today              : 2

Total VISITS (enqueues) today              : 6
Distinct patients ENQUEUED today           : 4
------------------------------------------------------------------------------------------
Hourly Enqueue Histogram:
Hour | Count | Bar
-----+-------+----------------------------------------
00   |     0 |
01   |     0 |
02   |     0 |
03   |     0 |
04   |     0 |
05   |     0 |
06   |     0 |
07   |     0 |
08   |     0 |
09   |     0 |
10   |     0 |
11   |     0 |
12   |     0 |
13   |     0 |
14   |     0 |
```

```
15   |     0 |
16   |     0 |
17   |     0 |
18   |     0 |
19   |     0 |
20   |     0 |
21   |     2 | ##
22   |     1 | #
23   |     3 | ###

Busiest hour: 23:00 (3 enqueues)
-----------------------------------------------------------------------------------------------
Gender (registrations): Male=3  Female=1  Unknown=0
Age groups (registrations): Children(<18)=1  Adults(18-64)=3  Seniors(65+)=0
Departments (by registration events):
  - vaccination               : 1
  - Cardiology                : 1
  - Physiotherapy             : 2

Appointment Types (registrations): Walk-in=2  Appointment=2  Unknown=0
Appointment Types (enqueues)     : Walk-in=4  Appointment=1
Registration ratio (Walk-in : Appointment) : 1.00 : 1
Enqueue ratio (Walk-in : Appointment)      : 4.00 : 1
===============================================================================================
                                        END OF THE REPORT
===============================================================================================
```

This report is a one-day snapshot of patient activity. At the top you see when the report was generated and which date it covers. “Total REGISTRATION events” counts every time a patient was registered that day; “Total DISTINCT patients registered” counts unique people. Then they are split into “New” (their first-ever registration date is that day) and “Returning” (same person registered again that day). “Total VISITS (enqueues)” counts how many times patients were put into the queue, and “Distinct patients ENQUEUED” counts the unique people who queued. The “Hourly Enqueue Histogram” shows how many enqueue actions happened each hour and highlights the busiest hour. The demographics section summarizes the registrations by gender and age bands. The department list shows how many registration events were recorded for each department. Finally, there are appointment-type breakdowns twice: once for registrations (Walk-in vs Appointment) and once for actual enqueues, followed by simple Walk-in:Appointment ratios for each.

```
===============================================================================================
                                   PATIENT QUEUE STATUS REPORT
===============================================================================================
Generated at: 2025-08-31 12:48:02
-----------------------------------------------------------------------------------------------
Current queue length                       : 2
Estimated average wait (currently waiting) : 4 minutes
Priority cases (emergencies) in queue      : 0
-----------------------------------------------------------------------------------------------
Daily Wait (last 7 days):
Date         | Served | Avg Wait (min)
-------------+--------+---------------
2025-08-25 |      0 |             0
2025-08-26 |      0 |             0
2025-08-27 |      0 |             0
2025-08-28 |      0 |             0
2025-08-29 |      0 |             0
2025-08-30 |      3 |             0
2025-08-31 |     13 |            54
-----------------------------------------------------------------------------------------------
Current Queue Composition:
Pos      Name                   Age   Department       Emergency  MinWait
--------+----------------------+-----+----------------+----------+----------
1        Ahmad Ali              30    Cardiology       no         9
2        Lim Wei                40    Pediatrics       no         0
===============================================================================================
                                        END OF THE REPORT
===============================================================================================
```

This queue status report shows the number of patients who are waiting in queue right now (Current queue length) and how long on average those still waiting have already been in line (Estimated average wait (currently waiting)). The Daily wait table shows the historical queue summary for the last 7 days. For each date it lists how many patients were served that day (Served) and the average time they actually waited from enqueue to service (Avg Wait (min)). Finally, Current Queue Composition lists everyone still in the queue in the order they will be served (emergencies first, then FIFO), showing their name, age, department, whether they're an emergency, and MinWait, which is how many minutes they've waited so far.

Name	Student ID	Prog / Tut.Grp	Signature
Sebastian Tan Huang Zhi	2400311	RDS2S2G1	THZ

Module name: Doctor Management Module

1. Source codes (Control classes)

```
package control;

/**
 *
 * @author Sebastian
 */
import adt.*;

import adt.SortedSkipList;
import entity.Doctor;
import entity.DutySchedule;
import boundary.DoctorPanel;
import dao.DoctorDAO;
import dao.DutyScheduleDAO;
import java.time.DayOfWeek;
import java.time.LocalTime;
import java.time.LocalDate;

public class MaintainDoctor {
    DutySchedule maxDuty = createMaxDuty();
    DutySchedule minDuty = createMinDuty();
    private ListInterface<Doctor> doctorList = new LinkedList<>();
    private DoctorDAO doctorDAO = new DoctorDAO();
    private DoctorPanel doctorPanel = new DoctorPanel();
    private SortedListInterface<DutySchedule> dutyScheduleList = new SortedSkipList<>(minDuty, maxDuty);
    private DutyScheduleDAO dutyScheduleDAO = new DutyScheduleDAO();

    public MaintainDoctor() {
        Doctor.loadLastId();
        doctorList = doctorDAO.retrieveFromFile();
        dutyScheduleList = dutyScheduleDAO.retrieveFromFile();
    }

    public void runDoctorMaintenance() {
        String choice = "0";
        do {
            choice = doctorPanel.getMainMenuChoice();
            switch (choice) {
```

```
            case "1": //manage doctor information
                runDoctorInfoManagement();
                break;
            case "2": //manage duty schedule
                dutyScheduleMenu();
                break;
            case "3": //manage availability
                runAvailabilityManagement();
                break;
            case "4": //generate report
                runReports();
                break;
            case "0": //exit from the program
                System.out.println("Exiting Doctor Management System.");
                break;
            default:
                System.out.println("Invalid option. Please try again.");
        }
    } while (!choice.equals("0"));
}

public void runDoctorInfoManagement() {
    String choice = "0";
    do {
        choice = doctorPanel.getManageDoctorChoice();
        switch (choice) {
            case "1": //add a new doctor to the system
                boolean added = addNewDoctor();
                if (added) {
                    System.out.println("Doctor added successfully.");
                } else {
                    System.out.println("Error: Clashing IC number.");
                }
                break;
            case "2": //list all the doctors in the system
                doctorPanel.viewAllDoctors(doctorList);
                break;
            case "3": //update the details of a doctor in the system
                doctorPanel.viewAllDoctors(doctorList);
                boolean updated = updateDoctor();
                if (updated) {
                    System.out.println("Doctor updated successfully.");
                }
                break;
            case "4": //remove the details of a doctor in the system
                doctorPanel.viewAllDoctors(doctorList);
                boolean removed = removeADoctor();
                if (removed) {
```

```
                System.out.println("Doctor removed successfully.");
            } else {
                System.out.println("Error: Doctor does not exists.");
            }
            break;
        case "0": //return to main menu
            System.out.println("Returning to main menu.");
            break;
        default:
            System.out.println("Invalid option. Please try again.");
    }
  } while (!choice.equals("0"));
}

 public void dutyScheduleMenu() {
    String choice = "0";
    do {
        choice = doctorPanel.getDutyScheduleMenuChoice();
        switch (choice) {
            case "1": //assign duty schedule
                doctorPanel.viewAllDoctors(doctorList);
                boolean assigned = assignDutySchedule();
                if (assigned) {
                    System.out.println("Duty added to schedule successfully.");
                }
                break;
            case "2": //view duty schedule by doctor
                doctorPanel.viewAllDoctors(doctorList);
                String doctorId = doctorPanel.inputDoctorId();
                SortedSkipList<DutySchedule> dutyList = getDutySchedulesByDoctor(doctorId);
                doctorPanel.viewDutySchedulesByDoctor(dutyList);
                break;
            case "3": //view duty schedule by date
                LocalDate date = doctorPanel.inputDate();
                SortedSkipList<DutySchedule> dutyListDate = getDutySchedulesByDate(date);
                doctorPanel.viewDutySchedulesByDate(dutyListDate, doctorList);
                break;
            case "4": //remove duty schedule
                boolean removed = removeDutySchedule();
                if (removed) {
                    System.out.println("Duty removed successfully.");
                }
                break;
            case "0": //back to main menu
                System.out.println("Returning to main menu.");
                break;
```

```
            default:
                System.out.println("Invalid option. Please try again.");
        }
    } while (!choice.equals("0"));
}

public void runAvailabilityManagement() {
    String availabilityChoice = "0";
    do {
        availabilityChoice = doctorPanel.getAvailabilityMenuChoice();
        switch (availabilityChoice) {
            case "1": //add date
                doctorPanel.viewAllDoctors(doctorList);
                String doctorId = doctorPanel.getDoctorToUpdate();
                Doctor doctorToUpdate = getDoctorById(doctorId);
                if (doctorToUpdate != null) {
                    System.out.println("Available dates for " + doctorToUpdate.getName());
doctorPanel.displayFormattedAvailability(doctorToUpdate.getAvailability(),(SortedSkipList<DutySchedule>)dutyScheduleList, doctorId);
                    LocalDate newDate = doctorPanel.inputNewAvailability();
                    SortedSkipList<LocalDate> tempList = doctorToUpdate.getAvailability();
                    boolean added = tempList.add(newDate);
                    if (added) {
                        doctorToUpdate.setAvailability(tempList);
                        doctorDAO.saveToFile(doctorList);
                    } else {
                        System.out.println("Date already exists.");
                    }
                } else {
                    System.out.println("Doctor not found with ID: " + doctorId);  // ERROR MESSAGE
                }
                break;
            case "2": //view available dates
                doctorId = doctorPanel.getDoctorToUpdate();
                doctorToUpdate = getDoctorById(doctorId);
                if (doctorToUpdate != null) {
                    System.out.println("Available dates for " + doctorToUpdate.getName());
doctorPanel.displayFormattedAvailability(doctorToUpdate.getAvailability(),(SortedSkipList<DutySchedule>)dutyScheduleList, doctorId);
                } else {
                    System.out.println("Doctor not found with ID: " + doctorId);  // ERROR MESSAGE
                }
                break;
            case "3": //remove date
```

```
                doctorId = doctorPanel.getDoctorToUpdate();
                doctorToUpdate = getDoctorById(doctorId);
                if (doctorToUpdate != null) {  // ADD NULL CHECK
                    LocalDate aDate = doctorPanel.inputAvailabilityToRemove();
                    SortedSkipList<LocalDate> tempRemoveList = doctorToUpdate.getAvailability();
                    boolean exists = tempRemoveList.contains(aDate);
                    if (exists) {
                        tempRemoveList.remove(aDate);
                        doctorToUpdate.setAvailability(tempRemoveList);
                        doctorDAO.saveToFile(doctorList);
                    } else {
                        System.out.println("Date does not exists.");
                    }
                } else {
                    System.out.println("Doctor not found with ID: " + doctorId);  // ERROR MESSAGE
                }
                break;
            case "0": // return to main menu
                System.out.println("Returning to main menu.");
                break;
            default:
                System.out.println("Invalid option. Please try again.");
        }
    } while (!availabilityChoice.equals("0"));
}

public void runReports() {
    String choice = "0";
    do {
        choice = doctorPanel.getGenerateReportMenuChoice();
        switch (choice) {
            case "1": //Schedule coverage report
                generateScheduleCoverageReport();
                break;
            case "2": //Weekly roster table
                generateWeeklyRosterTable();
                break;
            case "0": //return to main menu
                System.out.println("Returning to main menu.");
                break;
            default:
                System.out.println("Invalid option. Please try again.");
        }
    } while (!choice.equals("0"));
}

```

```java
// ==================== DOCTOR MANAGEMENT ====================

public boolean addNewDoctor() {
    Doctor newDoctor = doctorPanel.inputDoctorDetails();
    String newDoctorIc = newDoctor.getIcNo();
    if (getDoctorByIc(newDoctorIc) == null) {//no existing doctor
        doctorList.add(newDoctor);
        doctorDAO.saveToFile(doctorList);
        Doctor.saveLastId();
        return true;
    } else {
        return false;
    }
}

public boolean removeADoctor() {
    String doctorId = doctorPanel.inputDoctorIdToRemove();
    Doctor doctorToRemove = getDoctorById(doctorId);
    if (doctorToRemove != null) {
        doctorList.remove(doctorToRemove);
        doctorDAO.saveToFile(doctorList);
        removeDutySchedule(doctorId);
        return true;
    } else {
        return false;
    }
}

public boolean updateDoctor() {
    String doctorId = doctorPanel.getDoctorToUpdate();
    Doctor doctorToUpdate = getDoctorById(doctorId);
    if (doctorToUpdate != null) {
        String choice = "0";
        do {
            choice = doctorPanel.getUpdateChoice();
            switch (choice) {
                case "1": //name
                    doctorPanel.changeDoctorName(doctorToUpdate);
                    doctorDAO.saveToFile(doctorList);
                    break;
                case "2": //ic
                    String newIc = doctorPanel.changeDoctorIc(doctorToUpdate);
                    if (getDoctorByIc(newIc) == null) {
                        doctorToUpdate.setIcNo(newIc);
                        doctorDAO.saveToFile(doctorList);
                    } else {
                        System.out.println("Error: Clashing IC no.");
                    }
```

```
                    break;
                case "3": //specialisation
                    doctorPanel.changeDoctorSpecialisation(doctorToUpdate);
                    doctorDAO.saveToFile(doctorList);
                    break;
                case "4": //contactNo
                    doctorPanel.changeDoctorContactNo(doctorToUpdate);
                    doctorDAO.saveToFile(doctorList);
                    break;
                case "5": //constultationFee
                    doctorPanel.changeDoctorConsultationFee(doctorToUpdate);
                    doctorDAO.saveToFile(doctorList);
                    break;
                case "0": //return to main menu
                    System.out.println("Returning to main menu");
                    return false;
                default:
                    System.out.println("Invalid option. Please try again.");
            }
        } while (!choice.equals("0"));
        return true;
    } else {
        System.out.println("Doctor ID does not exists.");
        return false;
    }
}

public Doctor getDoctorById(String doctorId) {
    boolean found = false;
    int index = 0;
    Doctor doctor = null;
    while (!found && index < doctorList.size()) {
        doctor = doctorList.get(index);
        if (doctor.getId().compareTo(doctorId) == 0) {
            found = true;
        }
        index++;
    }
    if (found) {
        return doctor;
    } else {
        return null;
    }
}

public Doctor getDoctorByIc(String ic) {
    boolean found = false;
    int index = 0;
```

```
        Doctor doctor = null;
        while (!found && index < doctorList.size()) {
            doctor = doctorList.get(index);
            if (doctor.getIcNo().compareTo(ic) == 0) {
                found = true;
            }
            index++;
        }
        if (found) {
            return doctor;
        } else {
            return null;
        }
    }

    public LinkedList<Doctor> findDoctorsBySpecialization(String specialization) {
        LinkedList<Doctor> listBySpecialisation = new LinkedList<>();
        for (int i = 0; i < doctorList.size(); i++) {
            Doctor doctor = doctorList.get(i);
            if (doctor.getSpecialisation().compareTo(specialization) == 0) {
                listBySpecialisation.add(doctor);
            }
        }
        return listBySpecialisation;
    }


    // ==================== DUTY SCHEDULE MANAGEMENT ====================

    public boolean assignDutySchedule() {
        String doctorId = doctorPanel.inputDoctorId();
        Doctor doctor = getDoctorById(doctorId);
        if (doctor == null) {
            System.out.println("Doctor not found with ID: " + doctorId);
            return false;
        }
        System.out.println("Available dates for " + doctor.getName());

doctorPanel.displayFormattedAvailability(doctor.getAvailability(),(SortedSkipList<DutySchedule>)dutyScheduleList, doctorId);
        LocalDate date = doctorPanel.inputDate();
        LocalTime startTime = doctorPanel.inputStartTime();
        LocalTime endTime = doctorPanel.inputEndTime(startTime);
        if (hasDutyScheduleConflict(date, startTime, endTime)) {
            System.out.println("Scheduling conflict detected for doctor " + doctorId);
            return false;
        }
```

```
        if (!doctorIsAvailable(doctor, date)) {
            System.out.println("Doctor is not available on this day.");
            return false;
        }
        DutySchedule newDuty = new DutySchedule(doctorId, date, startTime, endTime);
        dutyScheduleList.add(newDuty);
        dutyScheduleDAO.saveToFile(duty            ScheduleList);
        return true;
    }

    public boolean removeDutySchedule() {
        LocalDate date = doctorPanel.inputDate();
        LocalTime startTime = doctorPanel.inputStartTime();

        // Create a probe with the same doctor ID pattern that might exist
        DutySchedule probe = new DutySchedule("", date, startTime,
startTime.plusHours(1));

        // Use iterator to find the exact duty schedule
        try {
            boolean found = false;
            DutySchedule toRemove = null;

            // Iterate through all duty schedules to find the exact match
            for (DutySchedule duty : (Iterable<DutySchedule>) dutyScheduleList) {

                if (duty.getDate().equals(date) && duty.getStartTime().equals(startTime)) {
                    toRemove = duty;
                    found = true;
                    break;
                }
            }

            if (found && toRemove != null) {
                DutySchedule removed = dutyScheduleList.remove(toRemove);
                if (removed != null) {
                    dutyScheduleDAO.saveToFile(dutyScheduleList);
                    System.out.println("Duty schedule removed successfully.");
                    return true;
                } else {
                    System.out.println("Failed to remove duty schedule.");
                }
            } else {
                System.out.println("Duty schedule not found for " + date + " at " + startTime);
            }
        } catch (Exception e) {
            System.out.println("Error removing duty schedule: " + e.getMessage());
            e.printStackTrace();
```

```
        }

        return false;
    }

    public void removeDutySchedule(String doctorId) {
        for (DutySchedule duty: (Iterable<DutySchedule>)dutyScheduleList) {
            if (duty.getDoctorId().equals(doctorId)) {
                dutyScheduleList.remove(duty);
                dutyScheduleDAO.saveToFile(dutyScheduleList);
            }
        }
    }

    public SortedSkipList<DutySchedule> getDutySchedulesByDoctor(String doctorId)
{
        SortedSkipList<DutySchedule> result = new SortedSkipList<>(createMinDuty(),
createMaxDuty());

        try {
            for (DutySchedule duty : (Iterable<DutySchedule>) dutyScheduleList) {
                if (duty.getDoctorId().equals(doctorId)) {
                    result.add(duty);
                }
            }
        } catch (ClassCastException e) {
            System.out.println("Error: Duty schedule list contains wrong object types");
            System.out.println("Please delete duty_schedules.dat and restart the program");
            return new SortedSkipList<>(createMinDuty(), createMaxDuty());
        }

        return result;
    }

    public SortedSkipList<DutySchedule> getDutySchedulesByDate(LocalDate date) {
        //LinkedList<DutySchedule> allDuties = getAllDutySchedules();
        SortedSkipList<DutySchedule> result = new SortedSkipList<>(
                new DutySchedule("MIN", LocalDate.MIN, LocalTime.MIN, LocalTime.MIN),
                new DutySchedule("MAX", LocalDate.MAX, LocalTime.MAX,
LocalTime.MAX));

        try {
            for (DutySchedule duty : (Iterable<DutySchedule>) dutyScheduleList) {
                if (duty.getDate().equals(date)) {
                    result.add(duty);
                }
            }
        } catch (ClassCastException e) {
```

```
            System.out.println("Error: Duty schedule list contains wrong object types");
            System.out.println("Please delete duty_schedules.dat and restart the program");
            return new SortedSkipList<>(minDuty, maxDuty);
        }

        return result;
    }

    private boolean doctorIsAvailable(Doctor doctor, LocalDate date) {
        SortedSkipList<LocalDate> availabilityList = doctor.getAvailability();
        return (availabilityList.contains(date));
    }


    // ==================== REPORT GENERATION ====================

    public void generateScheduleCoverageReport() {
        doctorPanel.displayScheduleCoverageReport(doctorList, dutyScheduleList);
    }

    public void generateWeeklyRosterTable() {
        doctorPanel.displayWeeklyRosterTable(dutyScheduleList, doctorList);
    }

    public void generateDutyScheduleReport(LocalDate startDate, LocalDate endDate)
{
        System.out.println("=== DUTY SCHEDULE REPORT ===");
        System.out.println("Period: " + startDate + " to " + endDate);
        System.out.println("============================");

        for (LocalDate date = startDate; !date.isAfter(endDate); date = date.plusDays(1)) {
            DayOfWeek day = date.getDayOfWeek();
            System.out.println("\n" + day + " (" + date + "):");

            boolean hasDuties = false;
            for (DutySchedule duty : (Iterable<DutySchedule>)dutyScheduleList) {
                if (duty.getDay() == day) {
                    Doctor doctor = getDoctorById(duty.getDoctorId());
                    if (doctor != null) {
                        System.out.println("  " + duty.getStartTime() + " - " + duty.getEndTime() +
                                           ": " + doctor.getName());
                        hasDuties = true;
                    }
                }
            }

            if (!hasDuties) {
                System.out.println("  No duties scheduled");
```

```
        }
    }
}

// ==================== PRIVATE HELPER METHODS ====================

private boolean hasDutyScheduleConflict(LocalDate date, LocalTime startTime, LocalTime endTime) {
    DutySchedule probe = new DutySchedule("", date, startTime, endTime);
    return dutyScheduleList.contains(probe);
}

// ==================== SENTINEL CREATION METHODS ====================

private DutySchedule createMinDuty() {
    return new DutySchedule("MIN", LocalDate.MIN, LocalTime.MIN, LocalTime.MIN);
}

private DutySchedule createMaxDuty() {
    return new DutySchedule("MAX", LocalDate.MAX, LocalTime.MAX, LocalTime.MAX);
}


// ==================== UI INTEGRATION ====================

public static void main(String[] args) {
    MaintainDoctor maintainDoctor = new MaintainDoctor();
    maintainDoctor.runDoctorMaintenance();
}

private class Node {

    private Doctor data;
    private Node next;

    private Node(Doctor data) {
      this.data = data;
      this.next = null;
    }

    private Node(Doctor data, Node next) {
      this.data = data;
      this.next = next;
    }
}
```

```
}
```

2. Screenshots

```
=== ADD NEW DOCTOR ===
Enter doctor name: John
Enter IC number: 010101140001
Enter specialization: dermathology
Enter contact number: 0123456789
Enter consultation fee: 50
Doctor added successfully.
```

The add doctor function allows the user to add enter information of a new doctor into the system.

```
=== DOCTOR INFORMATION MANAGEMENT ===
1. Add Doctor
2. View All Doctors
3. Update Doctor
4. Remove Doctor
0. Back To Main Menu
Please choose an option: 2


=== ALL DOCTORS ===
Doctor Id  | Name                | IC Number       | Specialisation       | Contact No.     | Consultation Fee
-------------------------------------------------------------------------------------------------------------------
D0008      | John                | 010101140001    | dermathology         | 0123456789      | RM 50.00
D0009      | Ali                 | 010102140003    | cardiology           | 0123456780      | RM 50.00

Total doctors: 2

Press Enter to continue...
```

The view all doctors function outputs all of the doctor's information in the system in order of the doctor id.

```
=== UPDATE DOCTOR ===
Enter doctor ID to update: D0009
Select field to update:
1. Name
2. IC Number
3. Specialization
4. Contact Number
5. Consultation Fee
0. Return to previous page
Enter choice: 4
Enter new contact number: 0112223333
```

The update doctor function allows the user to input the doctor id of the doctor whose information is to be updated, the field to update, and the updated information. This will update the doctor's information in the system.

```
=== ALL DOCTORS ===
Doctor Id  | Name                | IC Number       | Specialisation       | Contact No.     | Consultation Fee
-------------------------------------------------------------------------------------------------------------------
D0008      | John                | 010101140001    | dermathology         | 0123456789      | RM 50.00
D0009      | Ali                 | 010102140003    | cardiology           | 0112223333      | RM 50.00

Total doctors: 2

Press Enter to continue...

=== REMOVE DOCTOR ===
Enter doctor ID to remove: D0009
Doctor removed successfully.
```

The remove doctor function takes as input the doctor id of the doctor to be removed from the system, and remove the respective doctor's information from the system.

```
Enter doctor ID: D0008
Available dates for John

+---------------------------------+
| SEPTEMBER 2025                  |
+---------------------------------+
|Sun  Mon  Tue  Wed  Thu  Fri  Sat|
+---------------------------------+
    [ 1] [ 2]   3  [ 4]   5    6
  7    8    9   10   11   12   13
 14   15   16   17   18   19   20
 21   22   23   24   25   26   27
 28   29   30
+---------------------------------+
[##] = Available (3 days)
[##] = On Duty (1 days)
 ##  = Not Available
Total available days: 3
Total duty days: 1
Enter date (yyyy-MM-dd): 2025-09-02
Enter start time (HH:mm): 11:00
Enter end time (HH:mm): 18:00
Duty added to schedule successfully.
```

The assign duty schedule function allows the user to assign a duty schedule for a doctor on the day they are available to work. The date, start time, and end time of the duty shift is input, and the duty schedule is updated.

```
Enter doctor ID: D0008

=== VIEW DUTY SCHEDULES BY DOCTOR ===
Duty schedules for doctor D0008:
2025-09-01 | 08:00 - 13:00
2025-09-02 | 11:00 - 18:00
Total duties: 2

Press Enter to continue...
```

The view duty schedules by doctor function allows the user to input the doctor id of the doctor whose duty schedule is needed. The output are the dates and times that the doctor will be on duty.

```
Enter date (yyyy-MM-dd): 2025-09-02

=== VIEW DUTY SCHEDULES BY DATE ===
Duty schedules for 2025-09-02:
Dr. Ali (cardiology) | 08:00 - 13:00
Dr. John (dermathology) | 11:00 - 18:00
Total duties: 2

Press Enter to continue...
```

The view duty schedules by date function allows the user to input the date. The output is the doctors and their duty time for that chosen date.

```
Enter date (yyyy-MM-dd): 2025-09-01
Enter start time (HH:mm): 08:00
Duty schedule removed successfully.
Duty removed successfully.
```

The remove duty schedule function takes as input the date, and start time of the duty slot to remove. If there is a slot that matches the input, it is removed from the system, and the doctor that previously was on duty for that slot will no longer be on duty for that slot.

```
=== UPDATE DOCTOR ===
Enter doctor ID to update: D0008
Available dates for John

+--------------------------------+
| SEPTEMBER 2025                 |
+--------------------------------+
|Sun  Mon  Tue  Wed  Thu  Fri  Sat|
+--------------------------------+
    [ 1] [ 2]   3  [ 4]   5    6
  7    8    9   10   11   12   13
 14   15   16   17   18   19   20
 21   22   23   24   25   26   27
 28   29   30
+--------------------------------+
[##] = Available (3 days)
[##] = On Duty (1 days)
 ##  = Not Available
Total available days: 3
Total duty days: 1
Enter new available date (yyyy-MM-dd): 2025-09-03
```

The add available date function allows the user to input the doctor id of the doctor to add a date they are available to work. The system will output the days that the doctor is available, and the days the doctor is on duty. The system will take as input a new available date that the doctor can work on, and the available date will be stored in the system.

```
=== UPDATE DOCTOR ===
Enter doctor ID to update: D0008
Available dates for John

+--------------------------------+
| SEPTEMBER 2025                 |
+--------------------------------+
|Sun  Mon  Tue  Wed  Thu  Fri  Sat|
+--------------------------------+
    [ 1] [ 2] [ 3] [ 4]   5    6
  7    8    9   10   11   12   13
 14   15   16   17   18   19   20
 21   22   23   24   25   26   27
 28   29   30
+--------------------------------+
[##] = Available (4 days)
[##] = On Duty (1 days)
 ##  = Not Available
Total available days: 4
Total duty days: 1
```

The view available date function will take as input the doctor id of the doctor. Then it outputs the days in which the doctor is available to work, and the days in which the doctor is on duty.

```
=== UPDATE DOCTOR ===
Enter doctor ID to update: D0008
Enter a date to remove (yyyy-MM-dd): 2025-09-04
```

The remove available date function will take as input the doctor id of the doctor and the date to be removed. If the matching date is found, the available date will be removed, and the doctor cannot be assigned to be on duty for the removed date.

```
=== DOCTOR SCHEDULE COVERAGE & CONFLICT REPORT ===
Period: Next 7 Days
Generated on: 2025-08-30
==================================================
Analysis Period: 2025-08-30 to 2025-09-05

DAILY COVERAGE SUMMARY:
-----------------------
Date                 | Doctors    | Total Hours     | Coverage Gaps
-------------------------------------------------------------------------------------------------------------------------
2025-08-30 (SAT)     | 0          | 0.0 hrs         | Missing: radiology, dermathology, cardiology
2025-08-31 (SUN)     | 0          | 0.0 hrs         | Missing: radiology, dermathology, cardiology
2025-09-01 (MON)     | 0          | 0.0 hrs         | Missing: radiology, dermathology, cardiology
2025-09-02 (TUE)     | 2          | 12.0 hrs        | Missing: radiology
2025-09-03 (WED)     | 0          | 0.0 hrs         | Missing: radiology, dermathology, cardiology
2025-09-04 (THU)     | 0          | 0.0 hrs         | Missing: radiology, dermathology, cardiology
2025-09-05 (FRI)     | 0          | 0.0 hrs         | Missing: radiology, dermathology, cardiology

POTENTIAL SCHEDULING ISSUES:
----------------------------
No scheduling conflicts detected.

DOCTOR SCHEDULE SUMMARY (Next 7 Days):
--------------------------------------
John                : 1 duties, 3 available days - OK
Ali                 : 1 duties, 2 available days - OK
Lim Kian Joo        : 0 duties, 2 available days - UNDERUTILIZED: Available but not scheduled
Ravi                : 0 duties, 3 available days - UNDERUTILIZED: Available but not scheduled

Press Enter to continue...
```

The schedule coverage report will show a report of the number of doctors working, total hours of work, and coverage gaps (lack of doctor with the stated specialisation on that day)

on each day for the following 7 days. Potential scheduling issues will be notified. A summary of the duty schedule will also be output. This shows the number of duties each doctor has for the following 7 days, and their number of available days. This report is meant to be used by the clinic to ensure that the scheduling is complete and duties are properly distributed amongst the doctor. It also ensures that no doctors are underutilised.

```
=== WEEKLY DUTY ROSTER ===
Week: 2025-08-30 to 2025-09-05
==========================
Doctor              | SAT (30)      | SUN (31)      | MON (1)       | TUE (2)       | WED (3)       | THU (4)       | FRI (5)       | Total
--------------------+---------------+---------------+---------------+---------------+---------------+---------------+---------------+--------
John                | -             | -             | -             | 11:00-18:00   | -             | -             | -             | 1 days
Ali                 | -             | -             | -             | 08:00-13:00   | -             | -             | -             | 1 days
Lim Kian Joo        | -             | -             | -             | -             | -             | -             | -             | 0 days
Ravi                | -             | -             | -             | -             | -             | -             | -             | 0 days
--------------------+---------------+---------------+---------------+---------------+---------------+---------------+---------------+--------
Doctors Working:    | 0 Drs         | 0 Drs         | 0 Drs         | 2 Drs         | 0 Drs         | 0 Drs         | 0 Drs         |

Legend: Green = On Duty, - = No Duty
        Blue = Doctor Total Days, Yellow = Doctors Per Day
```

The weekly summary report prints a duty roster for the next 7 days, showing the time of duty, number of doctors working each day, and number of days the doctor is on duty. This report is meant to be used by the receptionist in the clinic to inform patients on the doctors availability.

Name	Student ID	Prog / Tut.Grp	Signature
Low Zi Qing	2400272	RDS2S2G1	*LOW*

Module name: Consultation Management Module

1. Source codes (Control classes)

```
package control;

import adt.*;
import dao.ConsultationDAO;
import entity.*;
import java.util.Scanner;

/**
 *
 * @author Low Zi Qing
 */
public class MaintainConsultation {

    private ListInterface<Consultation> consultations;
    private ListInterface<Patient> patients;
    private ListInterface<Doctor> doctors;
    private Scanner sc = new Scanner(System.in);
    private ConsultationDAO consultationDAO;

    public MaintainConsultation(ListInterface<Patient> patients, ListInterface<Doctor>
doctors) {
        this.consultations = new LinkedList<>();
        this.patients = patients;
        this.doctors = doctors;
        this.consultationDAO = new ConsultationDAO();
        this.consultations = consultationDAO.retrieveFromFile();

        int maxId = 0;
        for (int i = 0; i < consultations.size(); i++) {
            String id = consultations.get(i).getId();
            int num = Integer.parseInt(id.substring(1));
            if (num > maxId) {
                maxId = num;
            }
        }
        Consultation.setCounter(maxId + 1);
    }

    // Adding new consultation
    public void addConsultation() {
        System.out.println("\n--- Add Consultation ---");
```

```java

        // Select patient
        System.out.println("\nSelect patient:");
        for (int i = 0; i < patients.size(); i++) {
            System.out.println((i + 1) + ". " + patients.get(i).getName());
        }
        int pChoice = getValidatedChoice(patients.size());
        Patient selectedPatient = patients.get(pChoice - 1);

        // Select doctor
        System.out.println("\nSelect doctor:");
        for (int i = 0; i < doctors.size(); i++) {
            System.out.println((i + 1) + ". " + doctors.get(i).getName());
        }
        int dChoice = getValidatedChoice(doctors.size());
        Doctor selectedDoctor = doctors.get(dChoice - 1);

        // Enter diagnosis
        String diagnosis;
        do {
            System.out.print("Enter diagnosis: ");
            diagnosis = sc.nextLine().trim();
            if (diagnosis.isEmpty()) {
                System.out.println("Diagnosis cannot be empty.");
            }
        } while (diagnosis.isEmpty());

        // Follow-up
        boolean followUp = false;
        String input;
        do {
            System.out.print("Is follow-up required? (yes/no): ");
            input = sc.nextLine().trim().toLowerCase();
            if (input.equals("yes")) {
                followUp = true;
                break;
            } else if (input.equals("no")) {
                followUp = false;
                break;
            } else {
                System.out.println("Invalid input! Enter 'yes' or 'no'.");
            }
        } while (true);

        Consultation consultation = new Consultation(selectedPatient, selectedDoctor,
diagnosis);
        consultation.setFollowUpRequired(followUp);
        consultations.add(consultation);
        consultationDAO.saveToFile(consultations);

        // Display added consultation in table
```

```
        ListInterface<Consultation> added = new LinkedList<>();
        added.add(consultation);
        System.out.println("\n--- Consultation Added ---");
        printConsultationsTable(added);
    }

    // View all consultations
    public void viewConsultations() {
        System.out.println("\n--- All Consultations ---");
        printConsultationsTable(consultations);
    }

    // ==================== UPDATE ====================
    public void updateConsultation() {
        viewConsultations();
        if (consultations.isEmpty()) {
            return;
        }

        System.out.print("Enter Consultation ID to update: ");
        String id = sc.nextLine().trim();

        Consultation consultation = null;
        for (int i = 0; i < consultations.size(); i++) {
            if (consultations.get(i).getId().equalsIgnoreCase(id)) {
                consultation = consultations.get(i);
                break;
            }
        }

        if (consultation == null) {
            System.out.println("Consultation ID not found!");
            return;
        }

        int option;
        do {
            System.out.println("\n--- Update Consultation ---");
            System.out.println("1. Update Diagnosis");
            System.out.println("2. Change Doctor");
            System.out.println("3. Change Patient");
            System.out.println("4. Toggle Follow-up Requirement");
            System.out.println("0. Finish Updating");
            System.out.print("Enter choice: ");

            option = getValidatedChoice(4, true); // allow 0
            switch (option) {
                case 1 -> {
                    System.out.print("Enter new diagnosis: ");
                    String diagnosis = sc.nextLine().trim();
                    if (!diagnosis.isEmpty()) {
```

```
                consultation.setDiagnosis(diagnosis);
            } else {
                System.out.println("Diagnosis cannot be empty.");
            }
        }
        case 2 -> {
            System.out.println("\nSelect new doctor:");
            for (int i = 0; i < doctors.size(); i++) {
                System.out.println((i + 1) + ". " + doctors.get(i).getName());
            }
            int choice = getValidatedChoice(doctors.size());
            consultation.setDoctor(doctors.get(choice - 1));
        }
        case 3 -> {
            System.out.println("\nSelect new patient:");
            for (int i = 0; i < patients.size(); i++) {
                System.out.println((i + 1) + ". " + patients.get(i).getName());
            }
            int choice = getValidatedChoice(patients.size());
            consultation.setPatient(patients.get(choice - 1));
        }
        case 4 ->
            consultation.setFollowUpRequired(!consultation.isFollowUpRequired());
        case 0 ->
            System.out.println("Finished updating consultation.");
    }
} while (option != 0);

ListInterface<Consultation> updated = new LinkedList<>();
updated.add(consultation);
System.out.println("\n--- Updated Consultation ---");
printConsultationsTable(updated);
consultationDAO.saveToFile(consultations);
}

// Delete consultation
public void deleteConsultation() {
    viewConsultations();
    if (consultations.isEmpty()) {
        return;
    }

    System.out.print("Enter Consultation ID to delete: ");
    String id = sc.nextLine().trim();

    Consultation toRemove = null;
    for (int i = 0; i < consultations.size(); i++) {
        if (consultations.get(i).getId().equalsIgnoreCase(id)) {
            toRemove = consultations.get(i);
            break;
        }
```

```
        }

        if (toRemove != null) {
            consultations.remove(toRemove);
            consultationDAO.saveToFile(consultations);
            ListInterface<Consultation> removed = new LinkedList<>();
            removed.add(toRemove);
            System.out.println("\n--- Deleted Consultation ---");
            printConsultationsTable(removed);
        } else {
            System.out.println("Consultation ID not found!");
        }
    }

    // Search consultations with various criteria
    public void searchByConsultationId(String consultationId) {
        ListInterface<Consultation> results = new LinkedList<>();
        for (int i = 0; i < consultations.size(); i++) {
            if (consultations.get(i).getId().equalsIgnoreCase(consultationId)) {
                results.add(consultations.get(i));
            }
        }
        System.out.println("\n--- Search Results for Consultation ID: " + consultationId + "
---");
        printConsultationsTable(results);
    }

    public void searchByPatientName(String patientName) {
        ListInterface<Consultation> results = new LinkedList<>();
        for (int i = 0; i < consultations.size(); i++) {
            if (consultations.get(i).getPatient().getName().equalsIgnoreCase(patientName)) {
                results.add(consultations.get(i));
            }
        }
        System.out.println("\n--- Search Results for Patient: " + patientName + " ---");
        printConsultationsTable(results);
    }

    public void searchByDoctorName(String doctorName) {
        ListInterface<Consultation> results = new LinkedList<>();
        for (int i = 0; i < consultations.size(); i++) {
            if (consultations.get(i).getDoctor().getName().equalsIgnoreCase(doctorName)) {
                results.add(consultations.get(i));
            }
        }
        System.out.println("\n--- Search Results for Doctor: " + doctorName + " ---");
        printConsultationsTable(results);
    }

    // Reports generation
    public void dailyReport() {
```

```
        System.out.println("\n--- Daily Consultation Report ---");
        ListInterface<ReportEntry> report = new LinkedList<>();

        for (int i = 0; i < consultations.size(); i++) {
            String dateKey = consultations.get(i).getDateTime().toLocalDate().toString();
            boolean found = false;

            for (int j = 0; j < report.size(); j++) {
                if (report.get(j).getPeriod().equals(dateKey)) {
                    report.get(j).increment();
                    found = true;
                    break;
                }
            }

            if (!found) {
                report.add(new ReportEntry(dateKey));
            }
        }

        if (report.isEmpty()) {
            System.out.println("No consultations recorded.");
            return;
        }

        System.out.printf("%-15s %-10s%n", "Date", "No. of Consultations");
        System.out.println("--------------------------------");
        for (int i = 0; i < report.size(); i++) {
            ReportEntry r = report.get(i);
            System.out.printf("%-15s %-10d%n", r.getPeriod(), r.getCount());
        }
    }

    public void weeklyReport() {
        System.out.println("\n--- Weekly Consultation Report ---");
        ListInterface<ReportEntry> report = new LinkedList<>();

        for (int i = 0; i < consultations.size(); i++) {
            String weekKey = String.valueOf(
                    consultations.get(i).getDateTime()
                            .get(java.time.temporal.IsoFields.WEEK_OF_WEEK_BASED_YEAR)
            );
            boolean found = false;
            for (int j = 0; j < report.size(); j++) {
                if (report.get(j).getPeriod().equals(weekKey)) {
                    report.get(j).increment();
                    found = true;
                    break;
                }
            }
            if (!found) {
```

```
            report.add(new ReportEntry(weekKey));
        }
    }

    if (report.isEmpty()) {
        System.out.println("No consultations recorded.");
        return;
    }

    System.out.printf("%-10s %-10s%n", "Week", "No. of Consultations");
    System.out.println("--------------------");
    for (int i = 0; i < report.size(); i++) {
        ReportEntry r = report.get(i);
        System.out.printf("%-10s %-10d%n", r.getPeriod(), r.getCount());
    }
}

public void monthlyReport() {
    System.out.println("\n--- Monthly Consultation Report ---");
    ListInterface<ReportEntry> report = new LinkedList<>();

    for (int i = 0; i < consultations.size(); i++) {
        String monthKey = consultations.get(i).getDateTime().getMonth() + " "
                + consultations.get(i).getDateTime().getYear();
        boolean found = false;

        for (int j = 0; j < report.size(); j++) {
            if (report.get(j).getPeriod().equals(monthKey)) {
                report.get(j).increment();
                found = true;
                break;
            }
        }

        if (!found) {
            report.add(new ReportEntry(monthKey));
        }
    }

    if (report.isEmpty()) {
        System.out.println("No consultations recorded.");
        return;
    }

    System.out.printf("%-15s %-10s%n", "Month", "No. of Consultations");
    System.out.println("-------------------------------");

    for (int i = 0; i < report.size(); i++) {
        ReportEntry r = report.get(i);
        System.out.printf("%-15s %-10d%n", r.getPeriod(), r.getCount());
    }
```

```
    }

    public void followUpReport() {
        System.out.println("\n--- Follow-up Appointment Report ---");
        ListInterface<Consultation> followUps = new LinkedList<>();
        for (int i = 0; i < consultations.size(); i++) {
            if (consultations.get(i).isFollowUpRequired()) {
                followUps.add(consultations.get(i));
            }
        }
        printConsultationsTable(followUps);
    }

    public void summaryReport() {
        int total = consultations.size();
        int followUps = 0;
        for (int i = 0; i < total; i++) {
            if (consultations.get(i).isFollowUpRequired()) {
                followUps++;
            }
        }

        System.out.println("\n--- Summary Report ---");
        System.out.printf("%-25s : %d%n", "Total consultations", total);
        System.out.printf("%-25s : %d%n", "Follow-up consultations", followUps);
        System.out.printf("%-25s : %d%n", "Routine consultations", (total - followUps));
    }

    // Print consultations in tabular format
    private void printConsultationsTable(ListInterface<Consultation> list) {
        if (list.isEmpty()) {
            System.out.println("No consultations found.");
            return;
        }

        System.out.printf("%-4s %-6s %-18s %-18s %-30s %-20s %-10s%n",
                "No.", "ID", "Patient", "Doctor", "Diagnosis", "Date & Time", "Follow-Up");
System.out.println("-----------------------------------------------------------------------------------------------------------------------");

        for (int i = 0; i < list.size(); i++) {
            Consultation c = list.get(i);
            System.out.printf("%-4s %-6s %-18s %-18s %-30s %-20s %-10s%n",
                    i + 1,
                    c.getId(),
                    c.getPatient().getName(),
                    c.getDoctor().getName(),
                    c.getDiagnosis(),
                    c.getDateTime().toLocalDate() + " " +
c.getDateTime().toLocalTime().withNano(0),
```

```
                c.isFollowUpRequired() ? "Yes" : "No");
        }
    }

    // Validation helpers
    private int getValidatedChoice(int max) {
        return getValidatedChoice(max, false);
    }

    private int getValidatedChoice(int max, boolean allowZero) {
        int choice = -1;
        while (true) {
            System.out.print("Enter choice" + (allowZero ? " (0-" + max + ")" : " (1-" + max + ")") + ": ");
            String input = sc.nextLine().trim();
            if (input.matches("\\d+")) {
                choice = Integer.parseInt(input);
                if ((allowZero && choice >= 0 && choice <= max) || (!allowZero && choice >= 1 && choice <= max)) {
                    break;
                }
            }
            System.out.println("Invalid input! Try again.");
        }
        return choice;
    }

}
```

2. Screenshots

```
=== Clinic Consultation Management ===
1. Add Consultation
2. View All Consultations
3. Search Consultation
4. Modify Consultation
5. Delete Consultation
6. Reports
0. Exit System
Enter choice: 1
```

Clinic Consultation Management Menu

```
Select patient:
1. Alice
2. Bob
3. Charlie
Enter choice (1-3): 1

Select doctor:
1. Dr. Smith
2. Dr. Johnson
3. Dr. Lee
Enter choice (1-3): 1
Enter diagnosis: Kidney Failure
Is follow-up required? (yes/no): yes
```

Users able to add new Consultation to the list by choosing available patients and doctors, key-in the diagnosis and decide whether they want to follow up

```
=== Clinic Consultation Management ===
1. Add Consultation
2. View All Consultations
3. Search Consultation
4. Modify Consultation
5. Delete Consultation
6. Reports
0. Exit System
Enter choice: 2

--- All Consultations ---
No.  ID     Patient            Doctor             Diagnosis                      Date & Time          Follow-Up
-----------------------------------------------------------------------------------------------------------------
1    C1     Alice              Dr. Smith          Kidney Failure                 2025-08-31 04:46:50  Yes
2    C2     Bob                Dr. Johnson        Fever and flu                  2025-08-31 04:49:29  No
3    C3     Charlie            Dr. Lee            Hypertension                   2025-08-31 04:50:19  Yes

Press Enter to continue...
```

Users able to view the existing consultation records

```
--- Search Consultation ---
1. By Consultation ID
2. By Patient Name
3. By Doctor Name
0. Back to Main Menu
Enter choice: 1
Enter Consultation ID: C1

--- Search Results for Consultation ID: C1 ---
No.  ID     Patient            Doctor             Diagnosis                      Date & Time          Follow-Up
-----------------------------------------------------------------------------------------------------------------
1    C1     Alice              Dr. Smith          Kidney Failure                 2025-08-31 04:46:50  Yes
```

Users able to search the existing record by using different attributes

Before:

```
--- All Consultations ---
No.  ID    Patient          Doctor          Diagnosis                    Date & Time          Follow-Up
-------------------------------------------------------------------------------------------------------------
1    C1    Alice            Dr. Smith       Kidney Failure               2025-08-31 04:46:50  Yes
2    C2    Bob              Dr. Johnson     Fever and flu                2025-08-31 04:49:29  No
3    C3    Charlie          Dr. Lee         Hypertension                 2025-08-31 04:50:19  Yes
Enter Consultation ID to update: C1

--- Update Consultation ---
1. Update Diagnosis
2. Change Doctor
3. Change Patient
4. Toggle Follow-up Requirement
0. Finish Updating
Enter choice: Enter choice (0-4): 1
Enter new diagnosis: Diabetes
```

After:

```
--- Updated Consultation ---
No.  ID    Patient          Doctor          Diagnosis                    Date & Time          Follow-Up
-------------------------------------------------------------------------------------------------------------
1    C1    Alice            Dr. Smith       Diabetes                     2025-08-31 04:46:50  Yes
```

Users able to modify the existing consultation records

Before:

```
--- All Consultations ---
No.  ID    Patient          Doctor          Diagnosis                    Date & Time          Follow-Up
-------------------------------------------------------------------------------------------------------------
1    C1    Alice            Dr. Smith       Diabetes                     2025-08-31 04:46:50  Yes
2    C2    Bob              Dr. Johnson     Fever and flu                2025-08-31 04:49:29  No
3    C3    Charlie          Dr. Lee         Hypertension                 2025-08-31 04:50:19  Yes
Enter Consultation ID to delete: C1

--- Deleted Consultation ---
No.  ID    Patient          Doctor          Diagnosis                    Date & Time          Follow-Up
-------------------------------------------------------------------------------------------------------------
1    C1    Alice            Dr. Smith       Diabetes                     2025-08-31 04:46:50  Yes

Press Enter to continue...
```

After:

```
--- All Consultations ---
No.  ID    Patient          Doctor          Diagnosis                    Date & Time          Follow-Up
-------------------------------------------------------------------------------------------------------------
1    C2    Bob              Dr. Johnson     Fever and flu                2025-08-31 04:49:29  No
2    C3    Charlie          Dr. Lee         Hypertension                 2025-08-31 04:50:19  Yes

Press Enter to continue...
```

User able to delete the existing consultation records

```
--- Consultation Reports ---
1. Daily Report
2. Weekly Report
3. Monthly Report
4. Follow-up Appointments Report
0. Back to Main Menu
Enter choice: 1
```

Users able to view different period report

```
--- Daily Consultation Report ---
Date            No. of Consultations
--------------------------------
2025-08-31      2

Press Enter to continue...
```

```
--- Monthly Consultation Report ---
Month           No. of Consultations
--------------------------------
AUGUST 2025     2

Press Enter to continue...
```

Example of daily & monthly consultation reports

```
--- Consultation Reports ---
1. Daily Report
2. Weekly Report
3. Monthly Report
4. Follow-up Appointments Report
0. Back to Main Menu
Enter choice: 4

--- Follow-up Appointment Report ---
No.  ID     Patient            Doctor             Diagnosis                    Date & Time          Follow-Up
-----------------------------------------------------------------------------------------------------------------
1    C3     Charlie            Dr. Lee            Hypertension                 2025-08-31 04:50:19  Yes

Press Enter to continue...
```

Users able to check the consultation that required follow up

Name	Student ID	Prog / Tut.Grp	Signature
Gan Jun Wei	2400372	RDS2S2G1	GJW

Module name: Medical Treatment Management Module

1. **Source codes (Control classes)**

```
package control;

import adt.ArrayList;
import dao.MedicineDAO;
import dao.TreatmentDAO;
import entity.Medicine;
import entity.Treatment;
import java.util.regex.Pattern;

public class MedicalTreatmentManagement {
    private final TreatmentDAO treatmentDAO;
    private final MedicineDAO medicineDAO;

    public MedicalTreatmentManagement() {
        treatmentDAO = TreatmentDAO.getInstance();
        medicineDAO = MedicineDAO.getInstance();
    }

    public boolean isValidPatientId(String patientId) {
        return patientId != null && Pattern.matches("^P\\d{4}$", patientId);
    }

    // FIXED: Updated pattern to match TRT prefix
    public boolean isValidTreatmentId(String treatmentId) {
        return treatmentId != null && Pattern.matches("^TRT\\d+$", treatmentId);
    }

    public boolean isValidMedicineId(String medicineId) {
        return medicineId != null && Pattern.matches("^M\\d{3}$", medicineId);
    }

    public boolean isValidDiagnosis(String diagnosis) {
        return diagnosis != null && Pattern.matches("^[a-zA-Z\\s\\-]{2,100}$",
diagnosis);
    }

    public Treatment createTreatment(String patientId, String diagnosis) {
        if (!isValidPatientId(patientId) || !isValidDiagnosis(diagnosis)) {
            System.out.println("Invalid patient ID or diagnosis!");
            return null;
        }
```

```
        String treatmentId = treatmentDAO.generateTreatmentId();
        Treatment treatment = new Treatment(treatmentId, patientId, diagnosis);
        treatmentDAO.addTreatment(treatment);
        return treatment;
    }

    public boolean addMedicineToTreatment(String treatmentId, String medicineId) {
        System.out.println("Adding medicine " + medicineId + " to treatment " +
treatmentId);

        if (!isValidTreatmentId(treatmentId)) {
            System.out.println("Invalid treatment ID: " + treatmentId);
            return false;
        }

        if (!isValidMedicineId(medicineId)) {
            System.out.println("Invalid medicine ID: " + medicineId);
            return false;
        }

        Treatment treatment = treatmentDAO.getTreatmentById(treatmentId);
        Medicine medicine = medicineDAO.getMedicineById(medicineId);

        if (treatment == null) {
            System.out.println("Treatment not found: " + treatmentId);
            return false;
        }

        if (medicine == null) {
            System.out.println("Medicine not found: " + medicineId);
            return false;
        }

        treatment.addMedicine(medicine);
        return treatmentDAO.updateTreatment(treatment);
    }

    public Treatment getTreatment(String treatmentId) {
        return treatmentDAO.getTreatmentById(treatmentId);
    }

    public ArrayList<Treatment> getPatientTreatments(String patientId) {
        return treatmentDAO.getTreatmentsByPatientId(patientId);
    }

    public ArrayList<Treatment> getAllTreatments() {
        return treatmentDAO.getAllTreatments();
    }

    public double calculateTreatmentTotal(String treatmentId) {
```

```
        Treatment treatment = treatmentDAO.getTreatmentById(treatmentId);
        return treatment != null ? treatment.calculateTotalPrice() : 0;
    }

    public ArrayList<Medicine> getRecommendedMedicines(String diagnosis) {
        return medicineDAO.getRecommendedMedicines(diagnosis);
    }

    public ArrayList<Medicine> getAllMedicines() {
        return medicineDAO.getAllMedicines();
    }

    public boolean updateTreatmentNotes(String treatmentId, String notes) {
        Treatment treatment = treatmentDAO.getTreatmentById(treatmentId);
        if (treatment != null) {
            treatment.setDoctorNotes(notes);
            return treatmentDAO.updateTreatment(treatment);
        }
        return false;
    }

    public ArrayList<String> getAllDiagnoses() {
        return medicineDAO.getAllDiagnoses();
    }

}
```

2. Screenshots

```
=== Medical Treatment Management System ===

=== Main Menu ===
1. Create New Treatment
2. Add Medicine to Treatment
3. View Treatment Details
4. View Patient Treatment History
5. Calculate Treatment Total
6. Get Recommended Medicines for Diagnosis
7. View All Medicine Prices
8. Update Treatment Notes
9. Exit
Choose option: 1
```

User need to create the new treatment by using their patientID

```
=== Create New Treatment ===
Enter Patient ID (PXXXX): P0001
Available Diagnoses:
1. Fever
2. Common Cold
3. Flu
4. Headache
5. Migraine
6. Back Pain
7. Muscle Pain
8. Joint Pain
9. Cough
10. Sore Throat
11. Asthma
12. Allergy
13. Skin Rash
14. Stomach Pain
15. Heartburn
16. Diarrhea
17. Hypertension
18. Diabetes
19. Bacterial Infection
20. Fungal Infection
Select diagnosis number: 1
Created Treatment with ID: TRT001
```

After entering their patientID,there are some diagnoses available for them to select and then generate the unique treatmentID for further use.

```
=== Recommended Medicines for Fever ===
============================================================
| Medicine             Price       Description             |
============================================================
| Paracetamol          RM8.50      Pain and fever relief   |
| Ibuprofen            RM12.00     Anti-inflammatory pain  |
============================================================

Do you want to add all recommended medicines? (Y/N): y

=== Adding Recommended Medicines ===
Adding medicine M002 to treatment TRT001
Added: Paracetamol (RM8.50)
Adding medicine M003 to treatment TRT001
Added: Ibuprofen (RM12.00)

 Total Amount: RM20.50

Do you want to add doctor notes? (Y/N): y
Enter Doctor Notes (optional): n
Doctor notes updated successfully!
```

Now, the recommended medicines for each specified diagnosis will provide them what medicines they have to purchase. There are also provide the medicine's price and description for helping them to know the effect and sum up all the total. It also provided doctor notes for patients to know other details.

```
Choose option: 2

=== Add Medicine to Treatment ===
Enter Treatment ID (TRTXXX): TRT001

Current Treatment Summary:
----------------------
Treatment ID: TRT001
Patient ID: P0001
Diagnosis: Fever
Current Total: RM20.50

Available Medicines:
==============================================================================
| No | Medicine Name            | Price    | Description                    |
==============================================================================
| 1  | Antibiotic               | RM25.00  | For bacterial infections       |
| 2  | Paracetamol              | RM8.50   | Pain and fever relief          |
| 3  | Ibuprofen                | RM12.00  | Anti-inflammatory pain relief  |
| 4  | Aspirin                  | RM7.00   | Pain relief and blood thinner  |
| 5  | Antihistamine            | RM15.00  | For allergies and itching      |
| 6  | Cough Syrup              | RM18.00  | Relieves cough symptoms        |
| 7  | Decongestant             | RM10.50  | Clears nasal congestion        |
| 8  | Antacid                  | RM9.00   | Relieves heartburn and indi    |
| 9  | Antidiarrheal            | RM11.00  | Controls diarrhea              |
| 10 | Lozenges                 | RM6.50   | Sore throat relief             |
| 11 | Insulin                  | RM45.00  | Diabetes management            |
| 12 | Blood Pressure Med       | RM32.00  | Hypertension treatment         |
| 13 | Inhaler                  | RM28.00  | Asthma relief                  |
| 14 | Antifungal Cream         | RM14.00  | Treats fungal infections       |
| 15 | Muscle Relaxant          | RM20.00  | Relieves muscle pain           |
==============================================================================

Select medicine number: 10

Selected: Lozenges - RM6.50 (Sore throat relief)
Confirm adding this medicine? (Y/N): y
Adding medicine M010 to treatment TRT001
Medicine added successfully!
```

New total: RM27.00

Add other medicine to treatment

```
=== Main Menu ===
1. Create New Treatment
2. Add Medicine to Treatment
3. View Treatment Details
4. View Patient Treatment History
5. Calculate Treatment Total
6. Get Recommended Medicines for Diagnosis
7. View All Medicine Prices
8. Update Treatment Notes
9. Exit
Choose option: 3

=== View Treatment Details ===
Enter Treatment ID (TRTXXX): TRT001

 TREATMENT DETAILS
========================================================================
| Treatment ID                  : TRT001                              |
| Patient ID                    : P0001                               |
| Diagnosis                     : Fever                               |
| Date                          : Sun Aug 31 18:45:14 GMT+08:00 2025 |
========================================================================
| MEDICINE                       PRICE       DESCRIPTION              |
========================================================================
| Paracetamol                    RM8.50      Pain and fever           |
| Ibuprofen                      RM12.00     Anti-inflammato          |
| Lozenges                       RM6.50      Sore throat relief       |
========================================================================
| TOTAL PRICE                  : RM27.00                              |
========================================================================
| DOCTOR NOTES                 : n                                    |
========================================================================
```

Show treatment details (based on the previous purchase information and date & time)

```
=== Main Menu ===
1. Create New Treatment
2. Add Medicine to Treatment
3. View Treatment Details
4. View Patient Treatment History
5. Calculate Treatment Total
6. Get Recommended Medicines for Diagnosis
7. View All Medicine Prices
8. Update Treatment Notes
9. Exit
Choose option: 4

=== View Patient Treatment History ===
Enter Patient ID (PXXXX): P0001

 Treatment History for Patient P0001:
------------------------------------------------------------------------
TreatmentID  Date         Diagnosis            Medicines       Total
------------------------------------------------------------------------
TRT001       Sun Aug 31   Fever                3               RM27.00
```

Show treatment history and total of medicines and price

```
=== Main Menu ===
1. Create New Treatment
2. Add Medicine to Treatment
3. View Treatment Details
4. View Patient Treatment History
5. Calculate Treatment Total
6. Get Recommended Medicines for Diagnosis
7. View All Medicine Prices
8. Update Treatment Notes
9. Exit
Choose option: 5

=== Calculate Treatment Total ===
Enter Treatment ID (TRTXXX): TRT001
 Total price for treatment TRT001: RM20.50
```

Calculate the price based on the treatmentID

```
Choose option: 6

=== Get Recommended Medicines for Diagnosis ===
Available Diagnoses:
1. Fever
2. Common Cold
3. Flu
4. Headache
5. Migraine
6. Back Pain
7. Muscle Pain
8. Joint Pain
9. Cough
10. Sore Throat
11. Asthma
12. Allergy
13. Skin Rash
14. Stomach Pain
15. Heartburn
16. Diarrhea
17. Hypertension
18. Diabetes
19. Bacterial Infection
20. Fungal Infection
Select diagnosis number: 5

 Recommended medicines for Migraine:
================================================================
| Medicine             Price           Description             |
================================================================
| Ibuprofen            RM12.00     Anti-inflammatory pain      |
| Aspirin              RM7.00      Pain relief and blood       |
================================================================
| TOTAL COST           RM19.00                                 |
================================================================
```

Get recommended medicines for diagnoses

```
=== Main Menu ===
1. Create New Treatment
2. Add Medicine to Treatment
3. View Treatment Details
4. View Patient Treatment History
5. Calculate Treatment Total
6. Get Recommended Medicines for Diagnosis
7. View All Medicine Prices
8. Update Treatment Notes
9. Exit
Choose option: 7

=== View All Medicine Prices ===
==============================================================================
|                           ALL MEDICINE PRICES                              |
==============================================================================
| ID       Medicine Name        Price       Description                      |
==============================================================================
| M001     Antibiotic           RM25.00     For bacterial infections         |
| M002     Paracetamol          RM8.50      Pain and fever relief            |
| M003     Ibuprofen            RM12.00     Anti-inflammatory pain relief    |
| M004     Aspirin              RM7.00      Pain relief and blood thinner    |
| M005     Antihistamine        RM15.00     For allergies and itching        |
| M006     Cough Syrup          RM18.00     Relieves cough symptoms          |
| M007     Decongestant         RM10.50     Clears nasal congestion          |
| M008     Antacid              RM9.00      Relieves heartburn and indi      |
| M009     Antidiarrheal        RM11.00     Controls diarrhea                |
| M010     Lozenges             RM6.50      Sore throat relief               |
| M011     Insulin              RM45.00     Diabetes management              |
| M012     Blood Pressure Med   RM32.00     Hypertension treatment           |
| M013     Inhaler              RM28.00     Asthma relief                    |
| M014     Antifungal Cream     RM14.00     Treats fungal infections         |
| M015     Muscle Relaxant      RM20.00     Relieves muscle pain             |
==============================================================================
```

View the price for all medicines

```
=== Main Menu ===
1. Create New Treatment
2. Add Medicine to Treatment
3. View Treatment Details
4. View Patient Treatment History
5. Calculate Treatment Total
6. Get Recommended Medicines for Diagnosis
7. View All Medicine Prices
8. Update Treatment Notes
9. Exit
Choose option: 8

=== Update Treatment Notes ===
Enter Treatment ID (TRTXXX): TRT001
Enter Doctor Notes (optional): n
Doctor notes updated successfully!
```

Updated the treatment details

Name	Student ID	Prog / Tut.Grp	Signature
Soo Zu Zheng	2400773	RDS2S2G1	SZZ

Module name: Pharmacy Management Module

1. Source codes (Control classes)

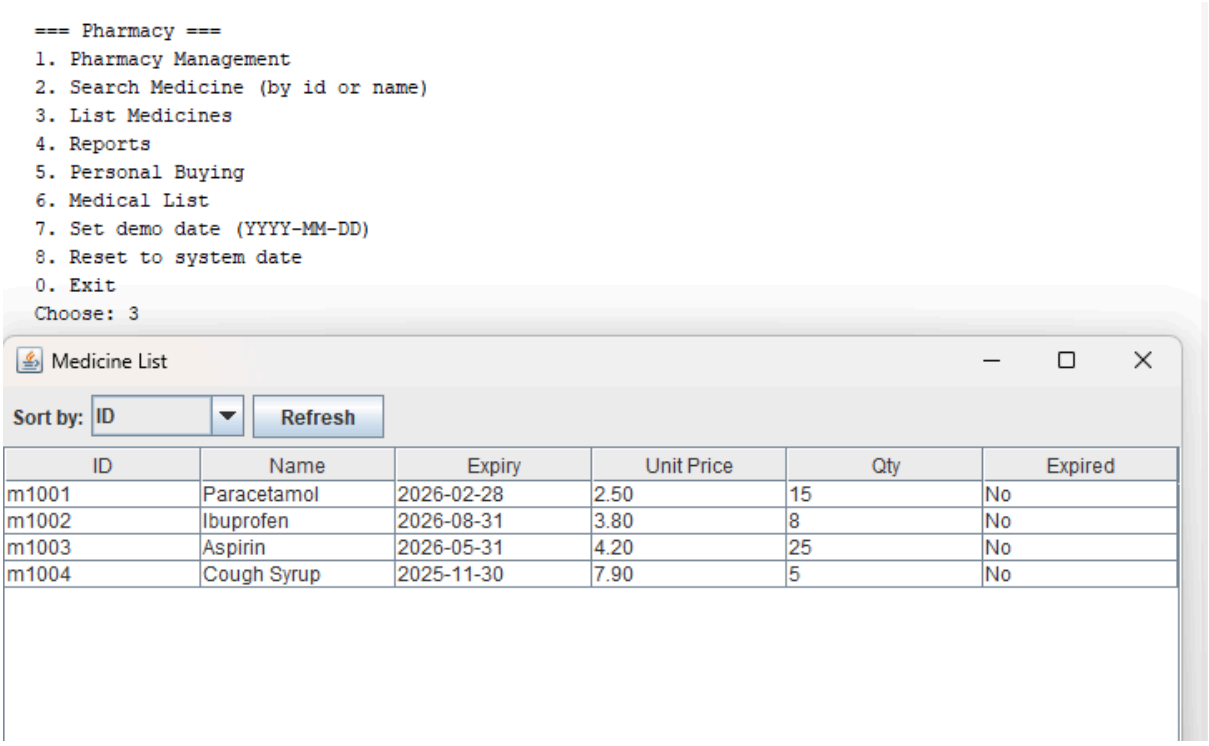
MedicineMaintenance.java

```
package control;

import adt.SortedLinkedList;
import adt.SortedListInterface;
import entity.Medicine;
import util.Comparer;
import util.IdGenerator;

import java.time.LocalDate;
/**
 *
 * @author SooZZ
 * @param <T>
 */

public class MedicineMaintenance {
    private final SortedListInterface<Medicine> list =
            new SortedLinkedList<>(new Comparer<Medicine>() {
                @Override public int compare(Medicine a, Medicine b) {
                    return a.getId().compareToIgnoreCase(b.getId());
                }
            });

    public Medicine add(String name, LocalDate expiryDate, double unitPrice, int quantity) {
        String id = IdGenerator.nextId(list.size());
        Medicine med = new Medicine(id, name, expiryDate, unitPrice, quantity);
        list.add(med);
        return med;
    }

    public boolean deleteById(String id) {
        int idx = indexOfId(id);
        if (idx < 0) return false;
        list.removeAt(idx);
        for (int i = idx; i < list.size(); i++) {
            Medicine m = list.get(i);
            int newNumber = 1000 + (i + 1);
            m.setId("m" + newNumber);
        }
        return true;
    }

    public Medicine findByIdOrNull(String id) {
        int idx = indexOfId(id);
        return idx < 0 ? null : list.get(idx);
    }

```

```
48  public Medicine[] searchByName(String namePart) {
49      String q = namePart == null ? "" : namePart.trim().toLowerCase();
50      int count = 0;
51      for (int i = 0; i < list.size(); i++) {
52          Medicine m = list.get(i);
53          if (m.getName().toLowerCase().contains(q)) count++;
54      }
55      Medicine[] out = new Medicine[count];
56      int k = 0;
57      for (int i = 0; i < list.size(); i++) {
58          Medicine m = list.get(i);
59          if (m.getName().toLowerCase().contains(q)) out[k++] = m;
60      }
61      return out;
62  }
63
64  public boolean updateQuantity(String id, int newQty) {
65      Medicine m = findByIdOrNull(id);
66      if (m == null) return false;
67      m.setQuantity(newQty);
68      return true;
69  }
70
71  public boolean updatePrice(String id, double newPrice) {
72      Medicine m = findByIdOrNull(id);
73      if (m == null) return false;
74      m.setUnitPrice(newPrice);
75      return true;
76  }
77
78  public Medicine[] listAll() {
79      Medicine[] out = new Medicine[list.size()];
80      for (int i = 0; i < list.size(); i++) out[i] = list.get(i);
81      return out;
82  }
83
84  public Medicine[] listAllSorted(Comparer<Medicine> by) {
85      Medicine[] arr = listAll();
86      for (int i = 1; i < arr.length; i++) {
87          Medicine key = arr[i];
88          int j = i - 1;
89          while (j >= 0 && by.compare(arr[j], key) > 0) {
90              arr[j + 1] = arr[j];
91              j--;
92          }
93          arr[j + 1] = key;
94      }
95      return arr;
96  }
97
98  public Medicine[] lowStock(int threshold) {
```

```
        int count = 0;
        for (int i = 0; i < list.size(); i++) if (list.get(i).getQuantity() < threshold) count++;
        Medicine[] out = new Medicine[count];
        int k = 0;
        for (int i = 0; i < list.size(); i++) {
            Medicine m = list.get(i);
            if (m.getQuantity() < threshold) out[k++] = m;
        }
        return out;
    }

    private int indexOfId(String id) {
        if (id == null) return -1;
        for (int i = 0; i < list.size(); i++) {
            if (id.equalsIgnoreCase(list.get(i).getId())) return i;
        }
        return -1;
    }
}
```

2. Screenshots



Initial medicine list

```
--- Pharmacy Management ---
1. Add Medicine
2. Delete Medicine
3. Update Quantity
4. Update Price
0. Back
Choose:
```

Pharmacy management submenu

```
--- Pharmacy Management ---
1. Add Medicine
2. Delete Medicine
3. Update Quantity
4. Update Price
0. Back
Choose: 1
Name: Antibiotics
Expiry (YYYY-MM-DD): 2025-11-20
Unit price: 5
Quantity: 20
Added: Medicine{id='m1005', name='Antibiotics', expiryDate=2025-11-20, unitPrice=5.0, quantity=20}
```

Add medicine function

```
--- Pharmacy Management ---
1. Add Medicine
2. Delete Medicine
3. Update Quantity
4. Update Price
0. Back
Choose: 2
ID to delete: m1001
Deleted.
```

ID	Name	Expiry	Unit Price	Qty	Expired
m1001	Ibuprofen	2026-08-31	3.80	8	No
m1002	Aspirin	2026-05-31	4.20	25	No
m1003	Cough Syrup	2025-11-30	7.90	5	No
m1004	Antibiotics	2025-11-20	5.00	20	No

Delete medicine function

```
--- Pharmacy Management ---
1. Add Medicine
2. Delete Medicine
3. Update Quantity
4. Update Price
0. Back
Choose: 3
ID: m1004
New quantity: 25
Updated.
```

Medicine List

Sort by: ID Refresh

ID	Name	Expiry	Unit Price	Qty	Expired
m1001	Ibuprofen	2026-08-31	3.80	8	No
m1002	Aspirin	2026-05-31	4.20	25	No
m1003	Cough Syrup	2025-11-30	7.90	5	No
m1004	Antibiotics	2025-11-20	5.00	25	No

Update quantity function

```
--- Pharmacy Management ---
1. Add Medicine
2. Delete Medicine
3. Update Quantity
4. Update Price
0. Back
Choose: 4
ID: m1004
New price: 100
Updated.
```

Medicine List

Sort by: ID Refresh

ID	Name	Expiry	Unit Price	Qty	Expired
m1001	Ibuprofen	2026-08-31	3.80	8	No
m1002	Aspirin	2026-05-31	4.20	25	No
m1003	Cough Syrup	2025-11-30	7.90	5	No
m1004	Antibiotics	2025-11-20	100.00	25	No

Update Price Function

```
=== Pharmacy ===
1. Pharmacy Management
2. Search Medicine (by id or name)
3. List Medicines
4. Reports
5. Personal Buying
6. Medical List
7. Set demo date (YYYY-MM-DD)
8. Reset to system date
0. Exit
Choose: 2
Search by (1) ID or (2) Name? 2
Name contains: u
Medicine{id='ml001', name='Ibuprofen', expiryDate=2026-08-31, unitPrice=3.8, quantity=8}
Medicine{id='ml003', name='Cough Syrup', expiryDate=2025-11-30, unitPrice=7.9, quantity=5}
```

```
=== Pharmacy ===
1. Pharmacy Management
2. Search Medicine (by id or name)
3. List Medicines
4. Reports
5. Personal Buying
6. Medical List
7. Set demo date (YYYY-MM-DD)
8. Reset to system date
0. Exit
Choose: 2
Search by (1) ID or (2) Name? 1
ID: ml004
Medicine{id='ml004', name='Antibiotics', expiryDate=2025-11-20, unitPrice=100.0, quantity=25}
```

Search by Id / Search by name

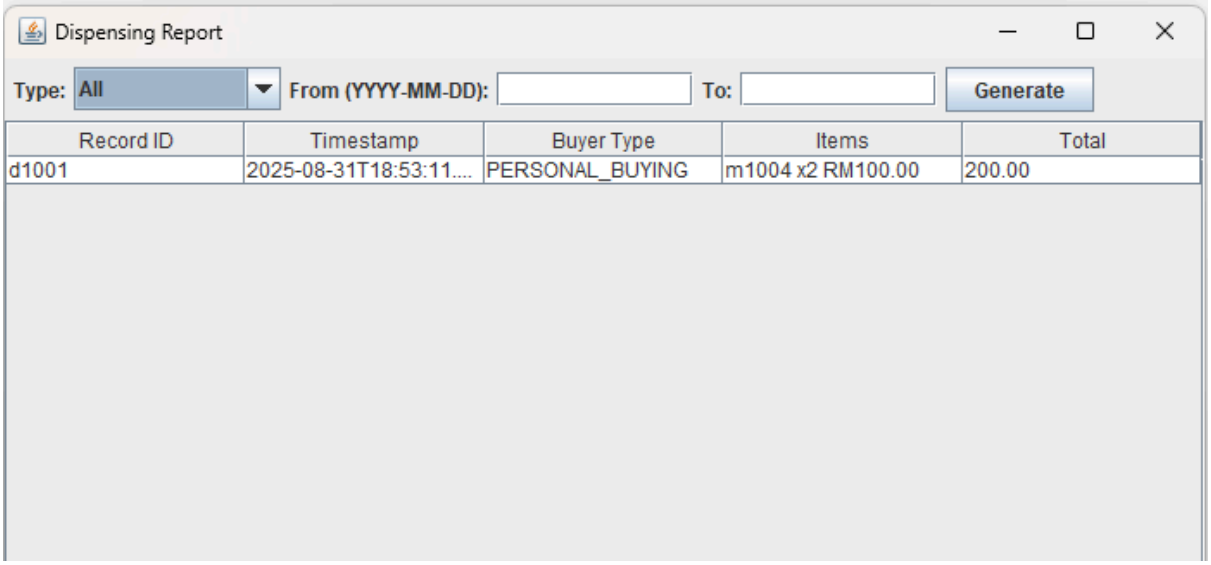
```
=== Pharmacy ===
1. Pharmacy Management
2. Search Medicine (by id or name)
3. List Medicines
4. Reports
5. Personal Buying
6. Medical List
7. Set demo date (YYYY-MM-DD)
8. Reset to system date
0. Exit
Choose: 5
Medicine ID (or blank to finish): ml004
Qty: 2
Added. Current total = RM 200.00
Medicine ID (or blank to finish):
Final total: RM 200.00
```

Personal walk in pharmacy

```
=== Pharmacy ===
1. Pharmacy Management
2. Search Medicine (by id or name)
3. List Medicines
4. Reports
5. Personal Buying
6. Medical List
7. Set demo date (YYYY-MM-DD)
8. Reset to system date
0. Exit
Choose: 4

--- Reports ---
1. Low Stock Report
2. Dispensing Report
0. Back
Choose: 1
```

Report 1: Low Stock Report

```
Medicine ID (or blank to finish): m1004
Qty: 2
Added. Current total = RM 200.00
Medicine ID (or blank to finish):
Final total: RM 200.00
```



Report2: Dispensing Report.

Actors:
Patient - register, edit personal information, arrange appointment
Doctor - register, edit personal information, manage schedule, manage diagnosis and treatments
Receptionist - retrieve patient record, manage consultation, maintain treatment history record
pharmacist - manage inventory, dispense medicine

Boundary CLASSES:
PatientRegistrationUI
DoctorManagementUI
ConsultationSchedulingUI
TreatmentEntryUI
PharmacyDispensingUI

Control CLASSES:
PatientManagement - manage patient registration, record maintenance and queuing management.
DoctorManagemenT - Manage doctor information, duty schedules and availability tracking.
ConsultationManagement - Manage patient consultations and arrange subsequent visit appointments.
MedicalTreatmentManagement - Manage patient diagnosis and maintain treatment history records
PharmacyManagement - Manage medicine dispensing after doctor consultation and maintain medicine stock control.

Entity CLASSES:
Patient (ID, name, contact, medical history, etc.)
Doctor (ID, name, specialty, schedule, availability)
Consultation (date, patient ID, doctor ID, notes)
Treatment (treatment ID, patient ID, diagnosis, prescribed medicines)
Medicine (medicine ID, name, quantity, expiry date)